

StructSynth: Dependency Graphs as Generation Plans for Low-Data Tabular Synthesis with Language Models

Anonymous ACL submission

Abstract

Tabular data derives its value from inter-feature dependencies, yet preserving them during synthesis is fragile when samples are scarce. Existing approaches either learn dependencies implicitly through distribution fitting, rely on statistical graph learning that becomes unstable with few samples, or encode structure through flat text serialization. Recent graph-aware methods use dependency graphs as attention biases or as backbones for non-LLM samplers, but they do not use the graph as a prompt-level plan for organizing the LLM’s own generation process. We introduce **StructSynth**, a framework that treats a dependency graph as a *generation plan*—determining the generation order, conditioning context, and scope of each black-box LLM call. In **Evidence-Grounded Graph Induction**, LLM reasoning and statistical association cues jointly construct a Directed Acyclic Graph (DAG) from limited samples. In **Graph-Planned Conditional Synthesis**, this DAG drives autoregressive synthesis in topological order, conditioning each feature on previously generated values with graph-specified parent structure guiding each step, making the conditioning schedule explicit throughout synthesis. Experiments show that **StructSynth** achieves state-of-the-art downstream utility and privacy preservation while maintaining competitive statistical fidelity, especially in low-data scenarios.

1 Introduction

Low-data tabular synthesis—generating realistic records from scarce samples in domains such as healthcare (Choi et al., 2017), finance (Rundo et al., 2019), and education (Luan and Tsai, 2021)—poses several intertwined challenges (Borisov et al., 2022). Among them, **preserving inter-feature dependencies** is particularly fragile under sample scarcity: the dependency relationships that enable Machine Learning (ML) models to learn meaningful decision boundaries (Fonseca and Bacao, 2023)

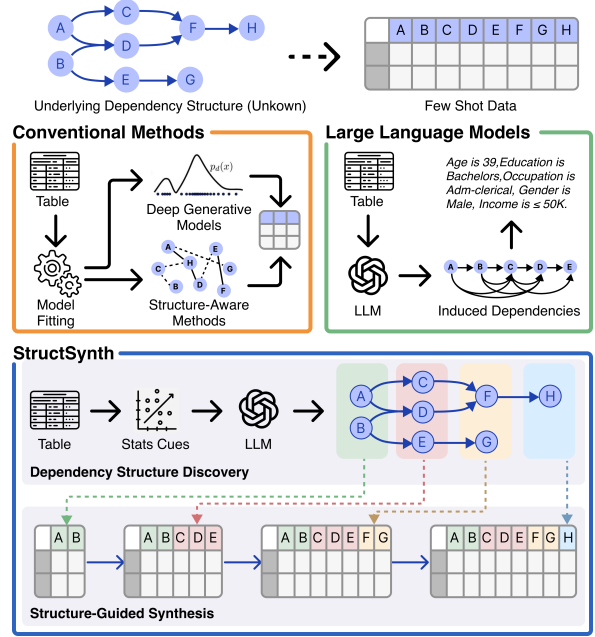


Figure 1: **Tabular synthesis paradigms under data scarcity.** Conventional methods learn dependencies implicitly via distribution fitting or through learned graphical structures; LLM-based methods encode them through serialized text. **StructSynth** discovers a dependency graph and treats it as a generation plan that determines the order, context, and scope of each LLM call.

rely on statistical signals that degrade rapidly as sample size shrinks. Without faithful dependencies, additional synthetic samples add volume without adding predictive signal (Liu et al., 2023).

As illustrated in Figure 1, existing tabular generators handle dependencies in three important but partial ways. **Deep generative models** capture the joint distribution end-to-end, implicitly learning inter-feature dependencies through distribution fitting; however, their performance varies across architectures, with many requiring substantially more samples to converge reliably. **Structure-aware methods** make dependencies explicit through graphical models such as Bayesian

networks (Ankan and Panda, 2015) and graph-based VAEs (Liu et al., 2023), yet the quality of the learned graph degrades under scarcity—errors in the discovered structure propagate directly into generation (Chai et al., 2022). **Large Language Models (LLMs)** exploit attribute names, descriptions, and in-context examples as semantic priors (Borisov et al., 2023; Seedat et al., 2024), but most LLM-based synthesizers encode dependencies only through flat textualization of attribute–value pairs, without explicitly modeling dependency directions or conditional independence structure (Liu et al., 2024).

Recent graph-aware methods take steps toward making dependencies explicit: GraDe (Zhang et al., 2025b) injects a learned dependency graph into the Transformer attention mask, while SPADA (Yang et al., 2025) uses the graph to guide normalizing flows. However, neither treats the graph as a *generation plan* that organizes the LLM’s own generation process—both use it as an internal modeling bias or an external sampler backbone, rather than as a prompt-level plan that determines what to generate, in what order, and with which context—and both require model fine-tuning or sufficient samples, leaving the low-data regime unaddressed. This gap raises a natural question: *can a dependency graph determine the generation order, conditioning context, and scope of each black-box LLM call?*

We answer this question with StructSynth, a framework for Structure-aware tabular data Synthesis that treats a dependency graph as a *generation plan*—one that determines how prompts are constructed and sequenced for black-box LLM synthesis. To use a graph as such a plan, two requirements must be met: the graph must be reliably discovered from limited samples, and it must organize the LLM’s generation process. This naturally leads to a two-stage design. In **Evidence-Grounded Graph Induction**, LLM-based reasoning is integrated with statistical association cues to construct a Directed Acyclic Graph (DAG)¹ from limited samples, with a cycle resolution mechanism ensuring acyclicity. Separating discovery from synthesis allows each stage to leverage complementary signals—semantic priors and statistical evidence—without being bottlenecked by a shared scarcity constraint.

¹We use “DAG” to denote a directed dependency graph used as a generative structure for organizing synthesis, not a ground-truth causal claim.

In **Graph-Planned Conditional Synthesis**, this DAG serves as the generation plan: synthesis proceeds autoregressively in topological order, with each layer of features conditioned on previously generated values and the local dependency sub-graph guiding parent-aware generation at every step. Our contributions are:

- We propose to treat a dependency graph as a **generation plan** that determines the generation order, conditioning context, and scope of each LLM call for tabular synthesis. We instantiate this idea in StructSynth, where **Evidence-Grounded Graph Induction** fuses LLM semantic priors with statistical association cues to discover reliable structures from limited samples, and **Graph-Planned Conditional Synthesis** compiles the discovered topology into a generation plan for layer-wise conditional synthesis.
- Experiments on six datasets show that StructSynth achieves state-of-the-art **downstream utility and privacy preservation** while maintaining competitive statistical fidelity. Ablations confirm that both stages contribute independently, validating the generation-plan design.

2 Related Work

Tabular Synthesis in Low-Data Regimes. Deep generative models—including VAE-based (Patki et al., 2016), GAN-based (Xu et al., 2019), and diffusion-based (Kotelnikov et al., 2023; Zhang et al., 2024) approaches—capture the joint distribution end-to-end but are data-hungry and provide limited control over inter-feature dependencies. Structure-aware methods address this by incorporating explicit dependency graphs; Bayesian-network-based generators (Ankan and Panda, 2015; Zhang et al., 2017) and graph-based VAEs (Van Breugel et al., 2021; Liu et al., 2023) condition synthesis on learned graphical structures, yet their effectiveness degrades when the underlying graphs cannot be reliably estimated from scarce samples.

LLM-Based Tabular Generation. Fine-tuning approaches such as GReaT (Borisov et al., 2023) and TAPTAP (Zhang et al., 2023) serialize rows as text and train LLMs for autoregressive generation, but are computationally expensive and degrade under data scarcity. Prompt-based methods avoid parameter updates: CuratedLLM (CLLM) (Seedat

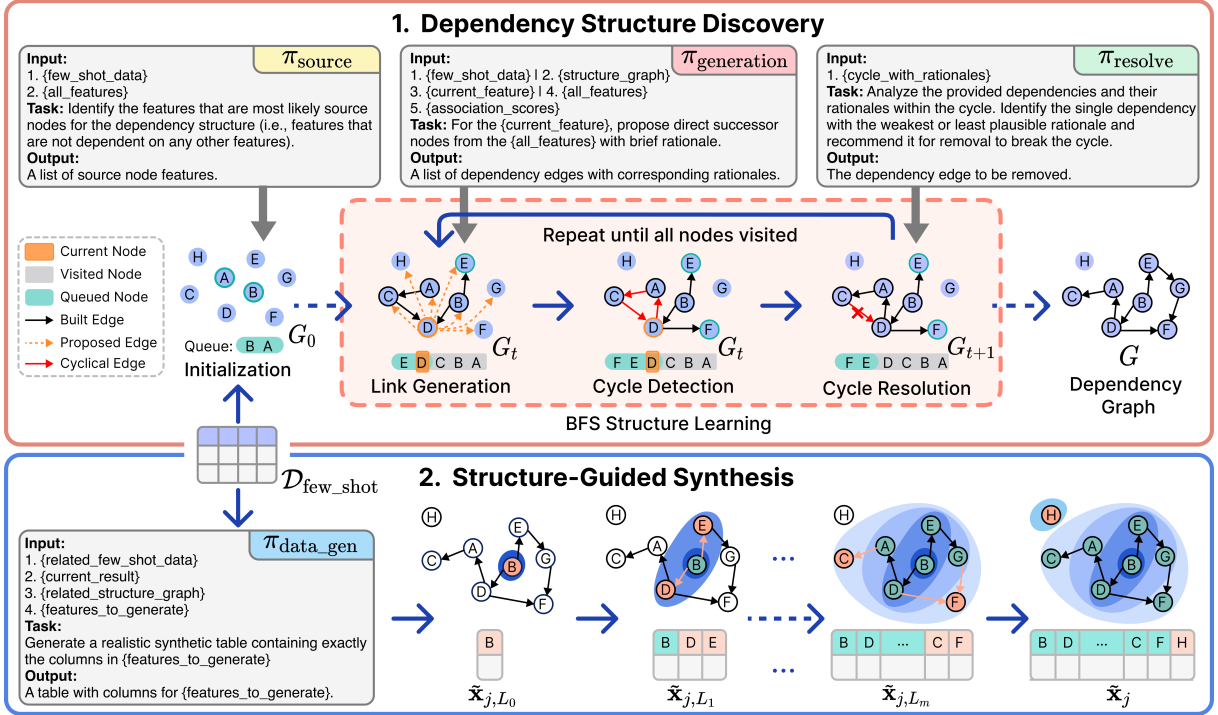


Figure 2: Overview of the StructSynth framework. The method first performs **Evidence-Grounded Graph Induction** to infer a dependency graph from limited data via an LLM-guided search. Subsequently, **Graph-Planned Conditional Synthesis** uses the learned graph as a generation plan to autoregressively guide the LLM in generating new tabular data.

et al., 2024) selects curated in-context examples to guide generation, yet does not explicitly model inter-feature dependency structure; TabGen-ICL (Fang et al., 2025) iteratively refines in-context selection to align the generated distribution with the real one. GraDe (Zhang et al., 2025b) moves closer to structure-aware LLM generation by injecting a learned sparse dependency graph into the Transformer attention mechanism, but the graph serves as an attention mask rather than an explicit generation plan, and requires model fine-tuning. SPADA (Yang et al., 2025) further decouples the two stages by replacing LLM-based generation with lightweight statistical estimators (KDE, normalizing flows), greatly accelerating sampling but relying on sufficient data to fit reliable conditional distributions.

LLM-Assisted Dependency Discovery. LLMs have been applied to structure learning itself. Early work demonstrated that LLMs can accurately judge pairwise causal directions from variable semantics alone (Kiciman et al., 2023). To scale beyond pairwise queries, BFS-based strategies reframe structure discovery as a graph traversal task, substantially reducing the number of LLM

calls (Jiralerspong et al., 2024; Zanna and Sano, 2025). A complementary line integrates LLM judgments with classical constraint-based algorithms, using the LLM to validate or correct statistically proposed edges (Ban et al., 2023). However, these methods target causal graph identification as an end in itself and do not connect the discovered structure to a downstream data generation process. An extended discussion of all the above lines of work is provided in Appendix A.

3 Methodology

Synthesizing realistic tabular data from scarce samples requires preserving the dependency relationships among features—the very relationships that enable downstream models to learn meaningful decision boundaries. Given a small tabular dataset $\mathcal{D}_{\text{train}} = (\mathbf{X}_{\text{train}}, \mathbf{A})$ with $n \leq 100$ instances and K attributes, our goal is to use an LLM-based generator f_{LLM} to produce a synthetic dataset $\mathcal{D}_{\text{synth}}$ that, when combined with $\mathcal{D}_{\text{train}}$, improves downstream model performance.

We address this by representing inter-feature dependencies as a Directed Acyclic Graph (DAG) and using it as a generation plan that organizes the LLM’s synthesis process. A DAG naturally

encodes generation order: its topological sort yields the synthesis sequence, and each node’s parents define the conditioning set (see Appendix C for further justification). Concretely, the graph determines three aspects of the generation plan: topological layers fix the generation order (§3.2), parent sets define the conditioning context, and local subgraphs bound the scope of each prompt. As depicted in Figure 2, StructSynth realizes this idea through two coupled stages. **Evidence-Grounded Graph Induction** constructs the DAG from limited data by combining LLM-based reasoning with statistical association cues—the LLM’s semantic prior acts as a structural regularizer while the statistical scores provide empirical grounding. **Graph-Planned Conditional Synthesis** then uses the learned DAG as a generation plan: synthesis proceeds autoregressively in topological order, conditioning each feature on previously generated values while the local dependency subgraph makes the planned parent structure explicit throughout generation.²

3.1 Evidence-Grounded Graph Induction

The dependency structure is represented as a DAG $G = (V, E)$, where the node set $V \subseteq \mathbf{A}$ contains selected attributes and edges E encode probabilistic dependencies. The graph is constructed iteratively via BFS as described below.

3.1.1 Graph Initialization

The graph construction begins by identifying source nodes—root attributes in the generation plan that are generated without conditioning on other attributes. Using a small subset $\mathcal{D}_{\text{few_shot}} \subseteq \mathcal{D}_{\text{train}}$ as in-context examples, the LLM determines the initial source nodes:

$$V_{\text{source}} = f_{\text{LLM}}(\pi_{\text{source}}(\mathcal{D}_{\text{few_shot}})). \quad (1)$$

The graph is initialized as $G_0 = (V_{\text{source}}, \emptyset)$ and expanded via BFS from these source nodes.

3.1.2 Expansion and Reasoned Link Generation

At iteration t , let $G_t = (V_t, E_t)$ denote the current graph. Each BFS step dequeues a node A_i and computes normalized association scores $\mathcal{S}(A_i) \in [0, 1]^{K-1}$ between A_i and all other attributes (Agresti and Kateri, 2011), using Cramér’s V for categorical pairs, $|r|$ (absolute

Pearson correlation) for continuous pairs, and the correlation ratio for mixed pairs (see Appendix D for details). The LLM is then queried with a prompt π_{generate} that integrates the current graph state G_t , the node A_i , and $\mathcal{S}(A_i)$ —pairing statistical evidence with the LLM’s semantic prior so that each compensates the other’s weakness at small n . The model proposes a set of direct successors for A_i , each accompanied by a textual rationale:

$$P_i = f_{\text{LLM}}(\pi_{\text{generate}}(A_i, G_t, \mathcal{S}(A_i), \mathcal{D}_{\text{few_shot}})). \quad (2)$$

From P_i , new directed edges and previously unseen nodes are extracted and added to the graph and BFS queue, respectively.

3.1.3 LLM-based Cycle Resolution

Integrating the proposed edges into G_t may introduce cycles. The LLM proposals yield a candidate edge set $E_{\text{prop},t}$ and a set of newly discovered attributes $V_{\text{new},t}$. A cycle detection algorithm identifies all elementary cycles Ψ in $E_t \cup E_{\text{prop},t}$. For each cycle $\psi \in \Psi$, the LLM is queried with π_{resolve} , which presents the conflicting edges together with their rationales. The model identifies the single least-justified edge for removal; the union of all removed edges forms E_{pruned} . The graph is then updated:

$$G_{t+1} = (V_t \cup V_{\text{new},t}, (E_t \cup E_{\text{prop},t}) \setminus E_{\text{pruned}}). \quad (3)$$

This expansion-resolution cycle continues until the BFS queue is empty, yielding the final DAG G .

3.2 Graph-Planned Conditional Synthesis

Upon learning the dependency structure $G = (V, E)$, we utilize it to guide synthetic tabular data generation. Constructing each synthetic data point $\tilde{\mathbf{x}}_j$ is performed in two sequential phases aligned with the underlying data structure.

3.2.1 Generating Graph-Based Values

The nodes V are partitioned into topological layers $\mathcal{L} = (L_1, \dots, L_m)$. For each layer L_i , we extract the dependency subgraph $G_i = G[L_i \cup \Gamma^-(L_i)]$, where $\Gamma^-(L_i)$ denotes parent nodes, along with the corresponding columns from the few-shot examples. Values for layer L_i are generated conditionally on all previously generated values $\tilde{\mathbf{x}}_{j,<i}$, with the local subgraph G_i specifying the parent dependencies to preserve:

$$\tilde{\mathbf{x}}_{j,L_i} = f_{\text{LLM}}\left(\pi_{\text{data_gen}}(\tilde{\mathbf{x}}_{j,<i}, G_i, L_i, \mathcal{D}_{\text{few_shot}}^i)\right). \quad (4)$$

²See Appendix K for full prompt templates and Algorithm 1 in Appendix E for the complete procedure.

Table 1: Comparison of Models on Downstream Model Performance (%). (C) and (R) denote classification and regression tasks, respectively. [†] indicates datasets created post LLM knowledge cutoff. Best results are in **bold**; second-best are underlined.

Type	Method	Adult (C)	Anxiety (C) [†]	Compas (C)	Salary (R) [†]	Obesity (R)	Churn (C)	Average	Avg. Rank
Real	$\mathcal{D}_{\text{train}}$	82.51 $\pm$ 1.93	85.49 $\pm$ 1.60	64.84 $\pm$ 2.51	49.71 $\pm$ 7.02	59.83 $\pm$ 6.40	86.86 $\pm$ 3.78	71.54	-
DGMs	TVAE	81.21 $\pm$ 1.54	79.63 $\pm$ 2.65	64.39 $\pm$ 2.06	46.18 $\pm$ 5.91	<u>60.92$\pm$2.61</u>	87.42 $\pm$ 3.80	69.96	7.00
	DDPM	81.72 $\pm$ 1.79	82.94 $\pm$ 3.24	66.01 $\pm$ 2.50	22.30 $\pm$ 15.70	43.20 $\pm$ 7.98	85.91 $\pm$ 2.05	63.68	8.67
	CTGAN	81.14 $\pm$ 1.81	76.06 $\pm$ 1.47	65.17 $\pm$ 2.61	46.17 $\pm$ 5.25	54.29 $\pm$ 5.90	86.50 $\pm$ 4.17	68.22	8.33
	NFlow	75.42 $\pm$ 4.58	74.83 $\pm$ 2.94	63.68 $\pm$ 4.22	25.55 $\pm$ 6.16	46.80 $\pm$ 4.17	84.99 $\pm$ 2.73	61.88	11.67
	TabSyn	83.88 $\pm$ 1.56	<u>85.68$\pm$2.40</u>	<u>67.77$\pm$6.98</u>	47.17 $\pm$ 9.58	43.50 $\pm$ 9.91	<u>89.86$\pm$1.53</u>	69.64	4.00
Structure Aware	GOGGLE	78.92 $\pm$ 3.04	80.92 $\pm$ 1.61	63.73 $\pm$ 3.85	43.18 $\pm$ 10.65	55.84 $\pm$ 7.77	85.27 $\pm$ 3.98	67.98	9.00
	BN	82.68 $\pm$ 1.35	82.77 $\pm$ 2.39	60.13 $\pm$ 3.46	46.33 $\pm$ 7.29	59.76 $\pm$ 3.46	88.85 $\pm$ 3.02	70.09	6.17
	DECAF	77.64 $\pm$ 3.91	79.90 $\pm$ 2.30	63.90 $\pm$ 3.69	43.58 $\pm$ 6.25	52.88 $\pm$ 6.44	83.94 $\pm$ 2.97	66.97	9.67
	SPADA-NF	81.25 $\pm$ 1.35	65.39 $\pm$ 2.64	65.58 $\pm$ 3.02	35.70 $\pm$ 5.70	45.73 $\pm$ 4.52	87.55 $\pm$ 1.24	63.53	8.83
LLMs	GReaT	82.55 $\pm$ 2.20	83.81 $\pm$ 2.55	64.02 $\pm$ 2.72	51.43 $\pm$ 3.31	47.03 $\pm$ 9.16	86.67 $\pm$ 3.20	69.25	6.33
	CLLM	<u>83.95$\pm$2.09</u>	85.17 $\pm$ 0.94	66.24 $\pm$ 1.51	<u>54.53$\pm$0.20</u>	60.43 $\pm$ 7.19	89.81 $\pm$ 2.78	<u>73.36</u>	<u>2.67</u>
	GraDe	83.72 $\pm$ 1.44	79.74 $\pm$ 2.01	66.15 $\pm$ 4.90	35.03 $\pm$ 8.08	22.92 $\pm$ 7.01	88.81 $\pm$ 2.51	62.73	7.67
Ours	StructSynth	85.55$\pm$1.08	86.45$\pm$0.57	69.40$\pm$0.25	55.98$\pm$2.96	62.27$\pm$5.40	90.39$\pm$1.96	75.01	1.00

Iterating through all layers yields a partial synthetic data point $\tilde{\mathbf{x}}_{j,V}$ comprising values for all graph-dependent attributes.

3.2.2 Completing Synthesis

Values for isolated attributes $A_{\text{iso}} = \mathbf{A} \setminus V$ (those outside the dependency graph) are generated in a single step, conditioned on all graph-based values $\tilde{\mathbf{x}}_{j,V}$:

$$\tilde{\mathbf{x}}_{j,A_{\text{iso}}} = f_{\text{LLM}}\left(\pi_{\text{iso}}(\tilde{\mathbf{x}}_{j,V}, A_{\text{iso}}, \mathcal{D}_{\text{few-shot}}^{\text{iso}})\right). \quad (5)$$

The graph-dependent and isolated values are concatenated to form each synthetic record; repeating s times yields $\mathcal{D}_{\text{synth}} = (\mathbf{X}_{\text{synth}}, \mathbf{A})$.

4 Experiments

4.1 Experimental Setups

Datasets. We conduct experiments on six real-world tabular datasets: Adult (Becker and Kohavi, 1996), Anxiety (Henderson et al., 2025), Compas (Angwin et al., 2016), Salary (Sajjad, 2025), Obesity (Palechor and de la Hoz Manotas, 2019), and Churn (Keramati et al., 2014). These datasets cover a range of domains (e.g., social, medical, business) and downstream tasks including binary/multi-class classification and regression. Notably, Anxiety and Salary were released after the knowledge cutoff of modern LLMs, allowing us to assess performance in the absence of prior exposure. Full dataset details are provided in Appendix F.1. Additionally, for evaluating structure discovery quality, we utilize three benchmark datasets from bnlearn (Scutari, 2010) with known ground-truth DAGs: Asia (Scutari, 1996a) (8

nodes), Child (Scutari, 1996b) (20 nodes), and Insurance (Scutari, 1996c) (27 nodes). These allow direct measurement of structural recovery via the Structural Hamming Distance (SHD) (de Jongh and Druzdzel, 2009).

Baselines. We compare our method against twelve baselines from three categories (implementation details in Appendix F.2): 1) **Deep Generative Models (DGMs)**, including TVAE (Xu et al., 2019), CTGAN (Xu et al., 2019), TabDDPM (Kotelnikov et al., 2023), NFlow (Papamakarios et al., 2021), and TabSyn (Zhang et al., 2024); 2) **Structure-Aware Methods**, including Bayesian Networks (Ankan and Panda, 2015), GOGGLE (Liu et al., 2023), DECAF (Van Breugel et al., 2021), and SPADA-NF (Yang et al., 2025); and 3) **LLM-based Methods**, including GReaT (Borisov et al., 2023), CLLM (Seedat et al., 2024), and GraDe (Zhang et al., 2025b).

Evaluation Metrics. Following previous work (Borisov et al., 2023; Seedat et al., 2024), we evaluate synthetic data along three axes (detailed definitions in Appendix F.4): **Downstream Model Performance** measures the utility of synthetic data by training a downstream model on $\mathcal{D}_{\text{aug}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{synth}}$ and evaluating on $\mathcal{D}_{\text{test}}$; we report AUC for classification and R^2 for regression. **Statistical Fidelity Error** quantifies how well pairwise correlations in the synthetic data match the real data (**lower is better**). **Privacy Risk** (van Breugel et al., 2023) measures whether synthetic records are overly similar to training records; the score is the fraction whose nearest

neighbor is drawn from $\mathcal{D}_{\text{train}}$ (**ideal: 0.5**).

Implementation Details. We use gpt-4o-mini with a temperature of 0.9 for both the CLLM and StructSynth models. Our experiments evaluate the low-data regime, generating 1000 synthetic samples ($s = 1000$) from 100 original samples ($n = 100$). Downstream performance is assessed using an XGBoost model. All experiments are repeated 10 times, and we report mean $\pm$ standard deviation. Unless otherwise stated, $\mathcal{D}_{\text{few_shot}} = \mathcal{D}_{\text{train}}$ in the 100-shot setting; Appendix F.3 lists the full hyperparameter configurations.

4.2 Main Results

Downstream Model Performance. Table 1 summarizes the downstream results. StructSynth achieves the best mean performance on all six datasets, with a perfect average rank of 1.00 and the highest average score (75.01). This improves over using only $\mathcal{D}_{\text{train}}$ (71.54) and surpasses the strongest competitor, CLLM (73.36), demonstrating that explicitly guiding an LLM with a graph-based generation plan yields substantial gains. Standard DGMs and structure-aware models perform notably weaker, as implicit distribution learning and data-driven graph discovery are unreliable in our $n = 100$ low-data setting.

Privacy Preservation and Statistical Fidelity Error. Table 2 reveals a fundamental tension between fidelity and privacy: high statistical fidelity often stems from memorization. For example, BN achieves the best fidelity (avg. rank 3.00) but exhibits poor privacy (avg. rank 9.00). GReaT ranks second in fidelity (avg. rank 4.00) yet has the worst privacy risk (90.15%), essentially reproducing training records. Similarly, SPADA-NF attains strong fidelity (avg. rank 4.50) at the cost of poor privacy (avg. rank 10.17). At the other end, prompt-based CLLM achieves strong privacy (avg. rank 3.50) but weaker fidelity (avg. rank 9.83). StructSynth achieves the best privacy preservation (avg. rank 1.50) while maintaining competitive statistical fidelity (avg. rank 8.50). Importantly, high pairwise fidelity does not guarantee downstream utility: the *Bayesian Sampler* ablation achieves the best raw fidelity (48.86) yet the lowest AUC (81.17). This is because Statistical Fidelity measures *pairwise* correlations, whereas StructSynth’s DAG encodes *conditional* dependencies between parent and child nodes—the relationships that actually drive downstream model

performance. The generation plan thus acts as a principled regularizer: it constrains overfitting to individual records while preserving the conditional structure that matters most for utility.

4.3 Ablation Study

To isolate the contributions of our method’s two main stages, we evaluate seven variants. For the structure learning stage: 1) *PC Discovery*, substituting our LLM-guided discovery with the classical Peter–Clark algorithm (Spirites et al., 2000); 2) *NoTears Discovery*, replacing our method with the continuous optimization-based NoTears algorithm (Zheng et al., 2018); 3) *No-Association Scores*, removing statistical association scores from the prompt to examine their guiding influence; and 4) *Pairwise Discovery*, replacing the efficient BFS traversal with less scalable pairwise queries to evaluate the performance-efficiency trade-off. For the data generation stage: 5) *No Topological Order*, conducting data generation based on the complete learned structure but ignoring the topological order; 6) *No Structure*, removing structural guidance entirely (equivalent to the CLLM); and 7) *Bayesian Sampler*, using a Bayesian sampler over the learned graph instead of the LLM generator.

The results (Table 3) yield three insights. First, **LLM-guided discovery outperforms classical alternatives**: StructSynth exceeds both *PC* and *NoTears* by 1.0–1.4 AUC pts, showing that semantic priors stabilize structure learning under data scarcity. Second, **structural guidance is multifaceted**: removing the graph (*No Structure*, −1.6 pts) or ignoring topological ordering (*No Topological Order*, −1.1 pts) both degrade utility. Finally, **synergy is vital**: replacing the LLM with a *Bayesian Sampler* causes the sharpest drop (−4.4 pts), confirming that the generation plan guides but cannot replace the LLM’s generative capabilities. Taken together, these results clarify why StructSynth’s advantage over CLLM is not merely incremental: CLLM processes features in an arbitrary serialized order, implicitly inducing dependencies from the input sequence, whereas StructSynth encodes dependencies as an explicit DAG so that relationships are *discovered* rather than incidentally induced.

The above gains generalize beyond the XGBoost evaluator used in our main experiments: across four downstream models (XGBoost, Random Forest, Logistic Regression/Ridge, MLP) on four representative datasets, StructSynth wins 14 out

Table 2: Comparison of Models on Privacy Preservation and Statistical Fidelity Error (%). [†] indicates datasets created post LLM knowledge cutoff. Best results are in **bold**; second-best are underlined.

Type	Method	Adult	Anxiety [†]	Compas	Salary [†]	Obesity	Churn	Average	Avg. Rank
Results for Privacy Preservation . Ranks are computed by deviation from 0.5, i.e., $ \text{PrivacyRisk} - 0.5 $; lower deviation indicates better privacy.									
DGMs	TVAE	50.34 $\pm$ 3.91	55.41 $\pm$ 4.26	64.50 $\pm$ 5.18	55.09 $\pm$ 4.38	52.93 $\pm$ 7.86	58.70 $\pm$ 4.75	56.16	4.50
	DDPM	99.95 $\pm$ 0.05	66.20 $\pm$ 13.17	67.18 $\pm$ 18.06	63.32 $\pm$ 19.52	71.36 $\pm$ 16.88	62.89 $\pm$ 24.91	71.82	10.00
	CTGAN	50.70 $\pm$ 3.64	56.64 $\pm$ 4.72	64.43 $\pm$ 4.22	53.59 $\pm$ 2.05	53.44 $\pm$ 3.89	57.84 $\pm$ 2.31	56.11	4.83
	NFlow	54.36 $\pm$ 1.92	53.52$\pm$3.08	62.57 $\pm$ 3.66	52.76 $\pm$ 2.34	51.74 $\pm$ 3.68	52.97 $\pm$ 3.33	54.65	3.67
	TabSyn	51.64 $\pm$ 2.67	70.53 $\pm$ 7.45	67.55 $\pm$ 5.40	54.66 $\pm$ 4.19	63.30 $\pm$ 7.19	59.38 $\pm$ 3.70	61.18	7.67
Structure Aware	GOGGLE	49.15 $\pm$ 7.97	43.62 $\pm$ 33.49	65.16 $\pm$ 17.25	30.94 $\pm$ 18.49	44.28 $\pm$ 23.83	61.87 $\pm$ 18.01	49.17	7.33
	BN	53.73 $\pm$ 2.36	72.98 $\pm$ 4.69	67.78 $\pm$ 3.63	61.50 $\pm$ 1.48	68.02 $\pm$ 2.18	59.68 $\pm$ 3.74	63.95	9.00
	DECAF	46.23 $\pm$ 19.13	58.59 $\pm$ 10.37	64.07 $\pm$ 19.14	56.91 $\pm$ 17.41	50.82 $\pm$ 15.49	69.79 $\pm$ 18.81	57.74	6.50
	SPADA-NF	95.20 $\pm$ 0.71	78.76 $\pm$ 15.53	57.86$\pm$11.26	94.98 $\pm$ 0.88	83.08 $\pm$ 1.69	92.30 $\pm$ 1.49	83.70	10.17
LLMs	GReaT	99.50 $\pm$ 0.35	85.39 $\pm$ 2.29	94.10 $\pm$ 1.33	86.66 $\pm$ 2.45	80.75 $\pm$ 2.99	94.51 $\pm$ 1.48	90.15	12.50
	CLLM	49.45 $\pm$ 3.85	44.37 $\pm$ 4.55	64.14 $\pm$ 4.22	<u>50.96$\pm$5.40</u>	45.36 $\pm$ 2.69	48.78$\pm$5.82	50.51	3.50
	GraDe	97.51 $\pm$ 1.60	61.14 $\pm$ 1.94	75.11 $\pm$ 5.11	65.99 $\pm$ 2.60	59.30 $\pm$ 4.76	69.07 $\pm$ 2.86	71.35	9.83
Ours	StructSynth	49.97$\pm$5.78	<u>44.74$\pm$5.13</u>	<u>62.37$\pm$3.86</u>	50.13$\pm$4.89	49.80$\pm$6.31	<u>48.67$\pm$4.84</u>	50.95	1.50
Results for Statistical Fidelity Error . A lower score indicates higher fidelity.									
DGMs	TVAE	53.71 $\pm$ 1.80	46.16 $\pm$ 1.28	60.26 $\pm$ 1.09	66.93 $\pm$ 1.13	58.41 $\pm$ 0.52	53.22 $\pm$ 1.54	56.45	6.50
	DDPM	65.26 $\pm$ 1.05	52.76 $\pm$ 1.22	64.94 $\pm$ 1.79	90.51 $\pm$ 0.33	64.44 $\pm$ 1.36	64.40 $\pm$ 2.90	67.05	11.33
	CTGAN	52.37 $\pm$ 1.56	47.14 $\pm$ 2.39	59.96 $\pm$ 0.90	58.57 $\pm$ 0.89	58.71 $\pm$ 0.57	52.32 $\pm$ 1.75	54.85	5.50
	NFlow	50.68 $\pm$ 2.71	43.50 $\pm$ 1.77	60.62 $\pm$ 0.93	52.38$\pm$1.10	58.85 $\pm$ 0.98	53.98 $\pm$ 2.24	<u>53.34</u>	5.50
	TabSyn	<u>49.17$\pm$1.34</u>	55.31 $\pm$ 1.31	57.23 $\pm$ 2.24	62.90 $\pm$ 3.01	58.08 $\pm$ 0.54	53.99 $\pm$ 2.08	56.11	6.17
Structure Aware	GOGGLE	68.32 $\pm$ 1.38	51.58 $\pm$ 7.27	66.19 $\pm$ 1.13	87.15 $\pm$ 1.65	63.80 $\pm$ 2.40	65.51 $\pm$ 2.16	67.09	11.33
	BN	47.46$\pm$2.15	38.57$\pm$1.01	60.33 $\pm$ 0.82	<u>54.39$\pm$1.52</u>	57.42 $\pm$ 0.75	<u>48.10$\pm$1.29</u>	51.05	3.00
	DECAF	62.71 $\pm$ 3.21	48.53 $\pm$ 1.53	63.82 $\pm$ 1.34	84.70 $\pm$ 2.41	60.93 $\pm$ 2.21	60.22 $\pm$ 0.88	63.49	9.50
	SPADA-NF	83.14 $\pm$ 0.73	45.92 $\pm$ 1.33	<u>53.62$\pm$1.55</u>	61.79 $\pm$ 0.55	55.29$\pm$0.85	47.85$\pm$1.17	57.94	4.50
LLMs	GReaT	82.71 $\pm$ 1.35	40.10 $\pm$ 1.64	52.17$\pm$1.18	58.75 $\pm$ 0.89	56.83 $\pm$ 0.87	48.18 $\pm$ 2.16	56.46	4.00
	CLLM	61.93 $\pm$ 4.43	57.64 $\pm$ 1.51	59.74 $\pm$ 2.33	78.60 $\pm$ 0.98	65.74 $\pm$ 1.42	62.20 $\pm$ 3.68	64.31	9.83
	GraDe	84.30 $\pm$ 0.86	42.59 $\pm$ 0.93	53.82 $\pm$ 2.62	55.85 $\pm$ 1.42	57.44 $\pm$ 1.43	53.66 $\pm$ 1.90	57.94	5.33
Ours	StructSynth	57.60 $\pm$ 3.13	57.86 $\pm$ 1.98	57.23 $\pm$ 1.69	64.64 $\pm$ 0.96	61.14 $\pm$ 1.10	58.63 $\pm$ 2.66	59.52	8.50

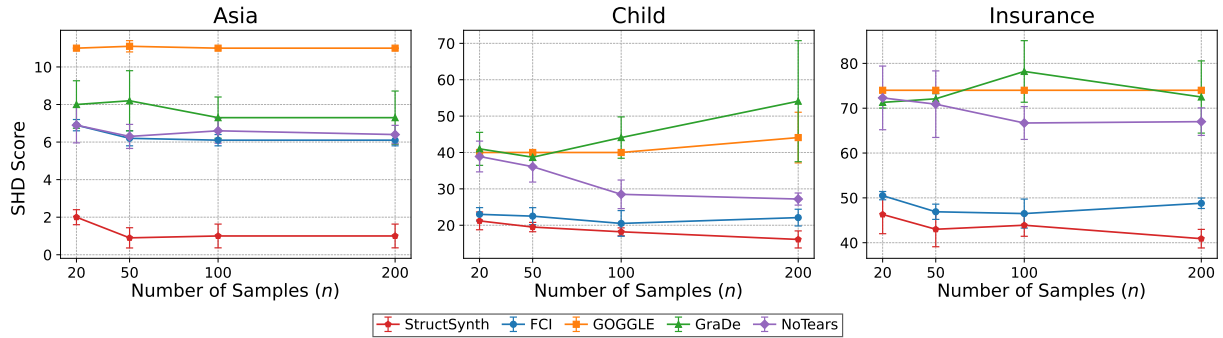


Figure 3: SHD of discovered dependency graphs vs. number of training samples on Asia, Child, and Insurance datasets with ground-truth DAGs. Lower SHD indicates better structural recovery.

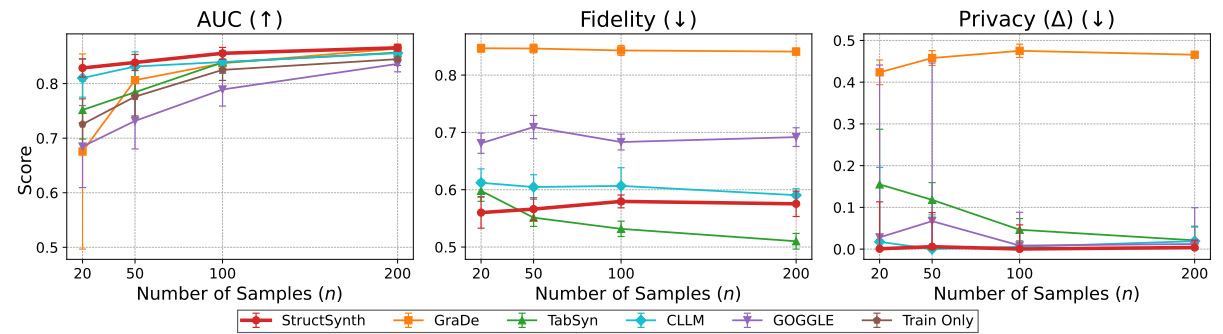


Figure 4: Effect of training sample size (n) on downstream utility (AUC $\uparrow$), statistical fidelity ($\downarrow$), and privacy risk deviation Δ ($\downarrow$) on the Adult dataset.

Table 3: Ablation Study on the Adult Dataset.

	Method	AUC	Fidelity	Privacy
	StructSynth	85.55	57.96	49.97
Structure Learning	PC Discovery	84.54	59.69	50.52
	NoTears Discovery	84.17	59.49	47.85
	No-Association Scores	84.06	59.10	46.60
	Pairwise Discovery	<u>85.25</u>	57.37	49.17
Generation	No Topological Order	84.41	60.47	51.03
	No Structure	83.95	60.67	49.45
	Bayesian Sampler	81.17	48.86	50.02

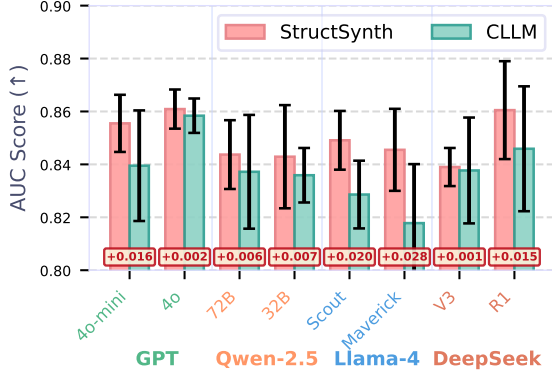


Figure 5: AUC on Adult across LLM backbones.

of 16 cells (Table 6 in Appendix H).

4.4 Structural Fidelity under Ground-Truth Graphs

We evaluate structure discovery quality on three bnlearn benchmarks with known ground-truth DAGs—Asia (8 nodes), Child (20 nodes), and Insurance (27 nodes)—using the Structural Hamming Distance (SHD), which counts the edge insertions, deletions, or reversals needed to recover the true graph (lower is better).

Figure 3 shows SHD across varying sample sizes ($n \in \{20, 50, 100, 200\}$). StructSynth achieves the lowest or near-lowest SHD on all three datasets, with the advantage most pronounced in low-data settings ($n \leq 50$) where baseline graph-learning methods such as FCI, GOGGLE, and NoTears produce substantially higher errors. This confirms that the LLM’s semantic prior effectively complements weak statistical signals, enabling robust structure recovery even from severely limited samples. Detailed graph visualizations are provided in Appendix G.

4.5 Influence of Training Sample Size

To evaluate data efficiency, we vary the training size n from 20 to 200 on the Adult dataset and measure AUC, Statistical Fidelity Error, and Privacy Risk

deviation $\Delta = |\text{PrivacyRisk} - 0.5|$ (Figure 4). StructSynth maintains high AUC, low fidelity error, and near-zero Δ across all sample sizes. The advantage is most pronounced in the low-data regime ($n \leq 50$), where baselines like BN and GOGGLE require significantly more samples to become competitive, while StructSynth sustains stable performance. This suggests that the generation plan acts as a regularizer: it prevents record-level overfitting and yields a stable utility-privacy-fidelity balance independent of sample size.

4.6 Influence of Different Language Models

To assess generalizability, we benchmark StructSynth against CLLM using a diverse set of open-source and proprietary LLMs on the Adult dataset (Figure 5). StructSynth outperforms CLLM across every tested architecture. The gain is most pronounced for models with moderate native capabilities (e.g., Maverick: +0.028 AUC), demonstrating that structural guidance can substantially boost weaker models. Even top-tier models like GPT-4o show measurable improvement, indicating that the benefit is complementary to model scale. Appendix I provides a token usage analysis showing that StructSynth incurs a modest +31% overhead over CLLM.

5 Conclusion

We presented StructSynth, a framework built on one core idea: treating a dependency graph as a generation plan that determines the order, conditioning context, and scope of each black-box LLM call for tabular synthesis. Separating graph induction from data generation allows each stage to leverage complementary signals—LLM semantic priors and statistical association cues for structure discovery, the discovered topology for graph-planned conditional synthesis. Experiments across six real-world datasets show that this generation-plan design achieves state-of-the-art downstream utility and privacy preservation while maintaining competitive statistical fidelity, with consistent gains across diverse LLM backends confirming the generalizability of the approach. More broadly, our results suggest that *explicit structural guidance* is a promising direction for tabular synthesis under data scarcity: rather than scaling model capacity or training data, investing in a reliable generation plan yields consistent improvements in generation quality.

Limitations

LLM semantic prior. StructSynth’s structure discovery quality depends on the LLM’s ability to interpret feature names and domain context. When schemas are opaque (e.g., anonymized identifiers such as V1, V2) or use highly specialized nomenclature (e.g., genomic locus codes), the semantic prior weakens considerably. Although statistical association cues can partially compensate, a fully anonymized schema effectively reduces our hybrid discovery to a purely statistical method, forfeiting the complementary benefit that motivates the design.

DAG assumption. The directed acyclic graph formulation, while well-suited for autoregressive generation, cannot directly represent cyclic or bidirectional dependencies. For data governed by genuine feedback loops—such as reciprocal economic indicators—the framework must approximate such relationships with unidirectional edges, potentially losing part of the underlying dependency structure.

LLM API dependence. The current implementation relies on commercial LLM APIs, raising cost and reproducibility concerns. While Appendix I reports modest token overhead and experiments across multiple LLM backends confirm robustness to the specific model choice, API version updates may still affect exact reproducibility of the discovered structures.

References

- Alan Agresti and Maria Kateri. 2011. Categorical data analysis. In *International encyclopedia of statistical science*, pages 206–208. Springer.
- Julia Angwin, Jeff Larson, Lauren Kirchner, and Surya Mattu. 2016. Machine Bias — [propublica.org](https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing). <https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing>.
- Ankur Ankan and Abinash Panda. 2015. pgmpy: Probabilistic graphical models using python. In *SciPy*, pages 6–11.
- Alessandro Antonucci, Gregorio Piqué, and Marco Zaffalon. 2023. Zero-shot causal graph extrapolation from text via llms. *arXiv preprint arXiv:2312.14670*.
- Vahan Arsenyan, Spartak Bughdaryan, Fadi Shaya, Kent Wilson Small, and Davit Shahnazaryan. 2024. Large language models for biomedical knowledge graph construction: Information extraction from emr notes. In *Proceedings of the 23rd Workshop on Biomedical Natural Language Processing*, pages 295–317.
- Taiyu Ban, Lyuzhou Chen, Derui Lyu, Xiangyu Wang, and Huanhuan Chen. 2023. Causal structure learning supervised by large language model. *arXiv preprint arXiv:2311.11689*.
- Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>.
- Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk, and Gjergji Kasneci. 2022. Deep neural networks and tabular data: A survey. *IEEE transactions on neural networks and learning systems*, 35(6):7499–7519.
- Vadim Borisov, Kathrin Sessler, Tobias Leemann, Martin Pawelczyk, and Gjergji Kasneci. 2023. [Language models are realistic tabular data generators](#). In *The Eleventh International Conference on Learning Representations*.
- Chengliang Chai, Jiayi Wang, Yuyu Luo, Zeping Niu, and Guoliang Li. 2022. Data management for machine learning: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(5):4646–4667.
- Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. 2002. Smote: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16:321–357.
- David Maxwell Chickering. 2002. Optimal structure identification with greedy search. *Journal of machine learning research*, 3(Nov):507–554.
- Edward Choi, Siddharth Biswal, Bradley Malin, Jon Duke, Walter F Stewart, and Jimeng Sun. 2017. Generating multi-label discrete patient records using generative adversarial networks. In *Machine learning for healthcare conference*, pages 286–305. PMLR.
- Martijn de Jongh and Marek J Druzdzel. 2009. A comparison of structural distance measures for causal bayesian network models. *Recent advances in intelligent information systems, challenging problems of science, computer science series*, pages 443–456.
- Liancheng Fang, Aiwei Liu, Hengrui Zhang, Henry Peng Zou, Weizhi Zhang, and Philip S. Yu. 2025. [TABGEN-ICL: Residual-aware in-context example selection for tabular data generation](#). In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20027–20041, Vienna, Austria. Association for Computational Linguistics.
- Joao Fonseca and Fernando Bacao. 2023. Tabular and latent space synthetic data generation: a literature review. *Journal of Big Data*, 10(1):115.

Liam Henderson, Ethan Callahan, and Charlotte Fraser. 2025. Social Anxiety Dataset — kaggle.com. https://www.kaggle.com/datasets/natezhang123/social-anxiety-dataset .	non small cell lung cancer. In <i>2024 IEEE 12th International Conference on Healthcare Informatics (ICHI)</i> , pages 688–693. IEEE.	694 695 696
Patrik Hoyer, Dominik Janzing, Joris M Mooij, Jonas Peters, and Bernhard Schölkopf. 2008. Nonlinear causal discovery with additive noise models. <i>Advances in neural information processing systems</i> , 21.	Fabio Mendoza Palechor and Alexis de la Hoz Manotas. 2019. Dataset for estimation of obesity levels based on eating habits and physical condition in individuals from colombia, peru and mexico . <i>Data in Brief</i> , 25:104344.	697 698 699 700 701
Thomas Jiralerspong, Xiaoyin Chen, Yash More, Vedant Shah, and Yoshua Bengio. 2024. Efficient causal graph discovery using large language models. <i>arXiv preprint arXiv:2402.01207</i> .	George Papamakarios, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. 2021. Normalizing flows for probabilistic modeling and inference. <i>Journal of Machine Learning Research</i> , 22(57):1–64.	702 703 704 705 706
A. Keramati, R. Jafari-Marandi, M. Aliannejadi, I. Ahmadian, M. Mozaffari, and U. Abbasi. 2014. Improved churn prediction in telecommunication industry using data mining techniques . <i>Applied Soft Computing</i> , 24:994–1012.	Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. 2016. The synthetic data vault. In <i>2016 IEEE international conference on data science and advanced analytics (DSAA)</i> , pages 399–410. IEEE.	707 708 709 710
Emre Kiciman, Robert Ness, Amit Sharma, and Chenhao Tan. 2023. Causal reasoning and large language models: Opening a new frontier for causality. <i>Transactions on Machine Learning Research</i> .	Judea Pearl. 2009. <i>Causality: Models, Reasoning, and Inference</i> , 2 edition. Cambridge University Press.	711 712
Daphne Koller and Nir Friedman. 2009. <i>Probabilistic Graphical Models: Principles and Techniques</i> . MIT Press.	Zhaozhi Qian, Bogdan-Constantin Cebere, and Mihaela van der Schaar. 2023. Synthcity: facilitating innovative use cases of synthetic data in different data modalities .	713 714 715 716
Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev, and Artem Babenko. 2023. Tabddpm: Modelling tabular data with diffusion models. In <i>International Conference on Machine Learning</i> , pages 17564–17579. PMLR.	Francesco Rundo, Francesca Trenta, Agatino Luigi Di Stallo, and Sebastiano Battiato. 2019. Machine learning for quantitative finance applications: A survey. <i>Applied Sciences</i> , 9(24):5574.	717 718 719 720
Tennison Liu, Zhaozhi Qian, Jeroen Berrevoets, and Mihaela van der Schaar. 2023. Goggle: Generative modelling for tabular data by learning relational structure. In <i>The Eleventh International Conference on Learning Representations</i> .	Bisma Sajjad. 2025. Global AI Job Market & Salary Trends 2025 — kaggle.com. https://www.kaggle.com/datasets/bismasajjad/global-ai-job-market-and-salary-trends-2025 .	721 722 723 724
Tianyang Liu, Fei Wang, and Muhao Chen. 2024. Rethinking tabular data understanding with large language models. In <i>Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)</i> , pages 450–482.	Marco Scutari. 1996a. Bayesian network repository: Asia (lung cancer) .	725 726
Hui Luan and Chin-Chung Tsai. 2021. A review of using machine learning approaches for precision education. <i>Educational Technology & Society</i> , 24(1):250–266.	Marco Scutari. 1996b. Bayesian network repository: Child .	727 728
Benjamin S Manning, Kehang Zhu, and John Horton. 2024. Automated social science: Language models as scientist and subjects. Technical report, National Bureau of Economic Research, Inc.	Marco Scutari. 1996c. Bayesian network repository: Insurance .	729 730
Narmada Naik, Ayush Khandelwal, Mohit Joshi, Madhusudan Atre, Hollis Wright, Kavya Kannan, Scott Hill, Giridhar Mamidipudi, Ganapati Srinivasa, Carlo Bifulco, and 1 others. 2024. Applying large language models for causal structure learning in	Marco Scutari. 2010. Learning bayesian networks with the bnlearn r package. <i>Journal of statistical software</i> , 35:1–22.	731 732 733
	Nabeel Seedat, Nicolas Huynh, Boris van Breugel, and Mihaela van der Schaar. 2024. Curated LLM: Synergy of LLMs and data curation for tabular augmentation in low-data regimes. In <i>Forty-first International Conference on Machine Learning</i> .	734 735 736 737 738
	Shohei Shimizu, Patrik O Hoyer, Aapo Hyvärinen, Antti Kerminen, and Michael Jordan. 2006. A linear non-Gaussian acyclic model for causal discovery. <i>Journal of Machine Learning Research</i> , 7(10).	739 740 741 742
	Alik Sokolov, Fabrizio Sabelli, Behzad Azadie Faraz, Wuding Li, and Luis Seco. 2024. Toward Automating Causal Discovery in Financial Markets and Beyond , chapter CHAPTER 14.	743 744 745 746

747	Peter Spirtes, Clark N Glymour, Richard Scheines, and	Mingming Zhang, Zhiqing Xiao, Guoshan Lu, Sai Wu,	801
748	David Heckerman. 2000. <i>Causation, prediction, and</i>	Weiqiang Wang, Xing Fu, Can Yi, and Junbo Zhao.	802
749	<i>search</i> . MIT press.	2025a. Aigt: Ai generative table based on prompt.	803
750	Peter Spirtes, Christopher Meek, and Thomas Richard-	In <i>Proceedings of the 31st International Conference</i>	804
751	son. 1995. Causal inference in the presence of	<i>on Computational Linguistics</i> , pages 9926–9938.	805
752	latent variables and selection bias. In <i>Proceedings of</i>	Tianping Zhang, Shaowen Wang, Shuicheng Yan,	806
753	<i>the Eleventh conference on Uncertainty in artificial</i>	Li Jian, and Qian Liu. 2023. Generative table pre-	807
754	<i>intelligence</i> , pages 499–506.	training empowers models for tabular prediction. In	808
755	Boris Van Breugel, Trent Kyono, Jeroen Berrevoets, and	<i>Proceedings of the 2023 Conference on Empirical</i>	809
756	Mihaela Van der Schaar. 2021. Decaf: Generating	<i>Methods in Natural Language Processing</i> , pages	810
757	fair synthetic data using causally-aware generative	14836–14854.	811
758	networks. <i>Advances in Neural Information</i>	Zheyu Zhang, Shuo Yang, Bardh Prenkaj, and Gjergji	812
759	<i>Processing Systems</i> , 34:22221–22233.	Kasneci. 2025b. Not all features deserve attention:	813
760	Boris van Breugel, Hao Sun, Zhaozhi Qian, and Mihaela	Graph-guided dependency learning for tabular data	814
761	van der Schaar. 2023. Membership inference attacks	generation with language models . In <i>Findings of the</i>	815
762	against synthetic data through overfitting detection.	<i>Association for Computational Linguistics: EMNLP</i>	816
763	In <i>International Conference on Artificial Intelligence</i>	2025, pages 6217–6242, Suzhou, China. Association	817
764	<i>and Statistics</i> , pages 3493–3514. PMLR.	for Computational Linguistics.	818
765	Feng Xie, Ruichu Cai, Biwei Huang, Clark Glymour,	Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and	819
766	Zhifeng Hao, and Kun Zhang. 2020. Generalized	Eric P Xing. 2018. Dags with no tears: Continuous	820
767	independent noise condition for estimating latent	optimization for structure learning. <i>Advances in</i>	821
768	variable causal graphs. In <i>NeurIPS</i> .	<i>neural information processing systems</i> , 31.	822
769	Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and	Yujia Zheng, Biwei Huang, Wei Chen, Joseph Ramsey,	823
770	Kalyan Veeramachaneni. 2019. Modeling tabular	Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter	824
771	data using conditional gan. <i>Advances in neural</i>	Spirtes, and Kun Zhang. 2024. Causal-learn: Causal	825
772	<i>information processing systems</i> , 32.	discovery in python. <i>Journal of Machine Learning</i>	826
773	Peter Xue-Kun Song. 2000. Multivariate dispersion	<i>Research</i> , 25(60):1–8.	827
774	models generated from gaussian copula. <i>Scandinavian</i>		
775	<i>Journal of Statistics</i> , 27(2):305–320.		
776	Shuo Yang, Zheyu Zhang, Bardh Prenkaj, and		
777	Gjergji Kasneci. 2025. Doubling your data in		
778	minutes: Ultra-fast tabular data generation via LLM-		
779	induced dependency graphs . In <i>Proceedings of the</i>		
780	<i>2025 Conference on Empirical Methods in Natural</i>		
781	<i>Language Processing</i> , pages 10337–10358, Suzhou,		
782	China. Association for Computational Linguistics.		
783	Khadija Zanna and Akane Sano. 2025. Fairness-driven		
784	llm-based causal discovery with active learning and		
785	dynamic scoring. <i>arXiv preprint arXiv:2503.17569</i> .		
786	Hengrui Zhang, Jiani Zhang, Balasubramaniam Sriniva-		
787	san, Zhengyuan Shen, Xiao Qin, Christos Faloutsos,		
788	Huzefa Rangwala, and George Karypis. 2024.		
789	Mixed-type tabular data synthesis with score-based		
790	diffusion in latent space. In <i>The twelfth International</i>		
791	<i>Conference on Learning Representations</i> .		
792	Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Di-		
793	vesh Srivastava, and Xiaokui Xiao. 2017. Privbayes:		
794	Private data release via bayesian networks. <i>ACM</i>		
795	<i>Transactions on Database Systems (TODS)</i> , 42(4):1–		
796	41.		
797	K Zhang and A Hyvärinen. 2009. On the identifiability		
798	of the post-nonlinear causal model. In <i>25th</i>		
799	<i>Conference on Uncertainty in Artificial Intelligence</i>		
800	<i>(UAI 2009)</i> , pages 647–655. AUAI Press.		

A Extended Related Work

This section provides an extended discussion of the related work summarized in Section 2, offering additional technical details on representative methods in each line of research.

A.1 Early and Statistical Methods

Early tabular data synthesis methods relied on statistical techniques. SMOTE (Chawla et al., 2002) generates synthetic minority-class samples by interpolating between nearest neighbors in feature space, addressing class imbalance but limited to local linear combinations. Copula-based methods (Xue-Kun Song, 2000) model the joint distribution by separately estimating marginal distributions and their dependency structure through copula functions, providing a principled statistical framework but struggling to capture complex, non-linear inter-feature relationships. These foundational approaches established the core objectives of tabular synthesis—realistic sample generation with dependency preservation—but their limited expressiveness motivated the shift toward deep generative models.

A.2 Deep Generative Models for Tabular Data

Deep generative models learn the data distribution through expressive neural architectures. TVAE (Patki et al., 2016) adapts the Variational Autoencoder framework to mixed-type tabular data by applying mode-specific normalization to continuous columns. CTGAN (Xu et al., 2019) introduces a conditional GAN with a training-by-sampling strategy that addresses the imbalanced categorical distributions common in tabular datasets. Normalizing Flow-based methods (Papamakarios et al., 2021) learn invertible transformations between a simple base distribution and the data distribution, offering exact likelihood computation. More recently, diffusion-based approaches have achieved strong results: TabDDPM (Kotelnikov et al., 2023) applies denoising diffusion probabilistic models to tabular data with separate handling of numerical and categorical features, while TabSyn (Zhang et al., 2024) first encodes mixed-type tabular data into a continuous latent space via a VAE and then applies score-based diffusion within that space, improving inter-column relation fidelity while reducing inference steps. Despite these advances, all DGM-based approaches learn feature dependencies only as a byproduct of distribution

fitting, offering limited interpretability or explicit control over the dependency structure during generation.

A.3 Structure-Aware Generative Methods

Structure-aware methods explicitly incorporate graphical models of feature dependencies into the generative process. Bayesian Network-based generators (Ankan and Panda, 2015) learn a directed acyclic graph over features and sample from the resulting factorized joint distribution. PrivBayes (Zhang et al., 2017) extends this framework with differential privacy guarantees by injecting calibrated noise into the learned conditional distributions, enabling privacy-preserving synthesis. Among deep-learning approaches, DECAF (Van Breugel et al., 2021) leverages a user-specified or learned causal graph to achieve fairness-aware generation by intervening on sensitive attributes, while GOGGLE (Liu et al., 2023) employs a graph-based VAE that jointly learns the dependency structure and the generative model in an end-to-end differentiable manner. However, the quality of these methods is fundamentally tied to the accuracy of the underlying graph, which degrades substantially under limited-sample settings where conditional independence tests lack statistical power and score-based search becomes prone to local optima. More recently, SPADA (Yang et al., 2025) employs an LLM to annotate sparse feature dependencies as a directed graph, then synthesizes data by traversing the graph with kernel density estimation or conditional normalizing flows, bypassing LLM-based generation entirely to achieve substantial sampling speedups.

A.4 LLM-Based Tabular Generation: Extended Discussion

Beyond the prompt-based and structure-aware LLM methods discussed in the main text, several fine-tuning approaches have been proposed. GReaT (Borisov et al., 2023) serializes each table row as a natural-language sentence with randomized column ordering and fine-tunes a pre-trained GPT-2 model for autoregressive row generation. TAPTAP (Zhang et al., 2023) introduces a table pre-training stage that exposes the LLM to diverse tabular datasets before task-specific fine-tuning, improving generalization across domains. AIGT (Zhang et al., 2025a) further augments the fine-tuning paradigm with attribute-

level instruction tuning, enabling the model to follow explicit generation directives for individual columns. While these fine-tuning methods can achieve high fidelity when sufficient training data is available, they incur substantial computational costs and their performance degrades markedly in low-data regimes ($n \leq 100$) where the limited training set is insufficient for effective parameter adaptation.

B Structure Learning Background

B.1 Conventional Structure Learning

Conventional approaches to dependency structure discovery primarily fall into two categories: constraint-based and score-based methods. Constraint-based methods, such as the Peter–Clark (PC) algorithm (Spirtes et al., 2000) and the Fast Causal Inference (FCI) algorithm (Spirtes et al., 1995), utilize conditional independence tests to iteratively prune edges from an initially fully connected graph. While PC assumes causal sufficiency (no hidden confounders), FCI is specifically designed to handle latent variables. In contrast, score-based methods optimize a global scoring criterion to identify the most suitable graph structure. A representative example is the Greedy Equivalence Search (GES) (Chickering, 2002), which searches through equivalence classes to maximize a scoring function. However, both constraint-based and score-based approaches typically yield only Markov equivalence classes, thereby failing to distinguish between structurally distinct graphs that encode identical conditional independencies. To address this limitation, Functional Causal Models (FCMs) impose stronger assumptions regarding the data generation process. FCMs model effects explicitly as functions of direct causes combined with independent noise terms, enabling the identification of causal directions since independence between cause and noise typically breaks down when reversed (Shimizu et al., 2006; Hoyer et al., 2008; Zhang and Hyvärinen, 2009). Recent advancements have generalized this approach, introducing conditions such as Generalized Independent Noise (GIN) to manage latent causal variables in linear, non-Gaussian settings (Xie et al., 2020). However, the statistical power of these traditional algorithms heavily relies on a large number of samples, making them unreliable for structure discovery in

the low-data regimes addressed by our work.

B.2 LLM-based Structure Learning

Leveraging Large Language Models (LLMs) for structure learning has recently emerged as a promising direction due to their inherent reasoning capabilities. Several paradigms have been proposed for integrating LLMs into structure discovery. Initially, methods focused on querying LLMs about pairwise causal relationships, showing strong accuracy in determining causal directions between variable pairs (Kiciman et al., 2023). To overcome the quadratic complexity inherent to pairwise approaches, more scalable methods employing breadth-first search (BFS) strategies have been introduced, reframing structure learning as a graph traversal task and significantly reducing the required number of queries to the LLM (Jiralerspong et al., 2024; Zanna and Sano, 2025). A third paradigm enhances reliability by integrating an iterative supervision loop, in which traditional structure-learning algorithms propose initial graphs that are subsequently validated or corrected by LLMs. This method grounds LLM-generated structures in empirical evidence, thereby reducing hallucination risks (Ban et al., 2023). LLM-based structure discovery methods have demonstrated notable effectiveness across diverse domains, including medicine (Naik et al., 2024; Antonucci et al., 2023; Arsenyan et al., 2024), economics (Sokolov et al., 2024), and social sciences (Manning et al., 2024). Inspired by the success of LLMs in leveraging prior knowledge for domain-specific reasoning, our work integrates a scalable, LLM-driven paradigm for robust structure discovery.

C Structural Formulation: Why a DAG?

A central design choice in StructSynth is the representation of inter-feature dependencies as a Directed Acyclic Graph (DAG). Since tabular feature relationships are often associative rather than inherently directional, this choice warrants explicit justification. We adopt the DAG formulation for three principled reasons.

Pragmatic Dependency Structure, Not Causal Claim. We emphasize that our DAG is a *generative dependency structure*: its edges encode statistical and probabilistic dependencies inferred from limited observations, not claims of ground-truth causality. This pragmatic interpretation

aligns with the foundational Bayesian Network (BN) literature (Pearl, 2009; Koller and Friedman, 2009), where DAGs serve as compact encodings of conditional independence relationships without necessarily asserting causal direction. The distinction is important: StructSynth uses the discovered DAG to *organize* the generation process, not to make ontological claims about the data-generating mechanism.

Functional Necessity of Directionality. The Graph-Planned Conditional Synthesis stage (Section 3.2) generates feature values autoregressively: each feature is produced conditioned on values already generated for its parent nodes. This autoregressive process inherently requires a *total ordering* over features to determine which values are available as conditioning context at each step. An undirected graph $\{A-B, B-C\}$ is ambiguous—it cannot specify whether A should be generated before C or vice versa. A DAG $\{A \rightarrow B \rightarrow C\}$ resolves this ambiguity by defining a topological order that directly translates into a layer-by-layer generation schedule. Even when the underlying statistical relationship between two features is symmetric (e.g., Age $\leftrightarrow$ Income), the generation process *must* select a direction—the DAG provides exactly this. The specific direction chosen does not affect *whether* the dependency is captured, only the *order* in which values are conditioned.

Acyclicity as a Mathematical Prerequisite. Beyond directionality, acyclicity is essential for the generation process to be well-defined. If cycles were permitted—e.g., $A \rightarrow B \rightarrow C \rightarrow A$ —the generation would encounter circular dependencies: the value of A would depend on C , which depends on B , which in turn depends on A , creating an unresolvable loop. The DAG constraint guarantees the existence of a valid topological ordering over nodes, ensuring that every feature can be generated exactly once, conditioned only on features whose values have already been determined.

Summary. In summary, the DAG in StructSynth serves a *functional* rather than *ontological* role: it imposes a well-defined, acyclic generation order that enables structure-conditioned autoregressive synthesis. This formulation is consistent with decades of probabilistic graphical model research and is a natural fit for our graph-as-plan formulation.

D Statistical Association Measures

To quantify the empirical dependencies between attribute pairs, the framework employs three association measures, chosen according to the data types of the variables involved.

D.1 Continuous-Continuous: Pearson’s R

For two continuous variables $x = \{x_i\}_{i=1}^n$ and $y = \{y_i\}_{i=1}^n$, Pearson’s correlation coefficient r is defined as the standardized covariance:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (6)$$

where $\bar{x}$ and $\bar{y}$ denote the sample means. The value of r ranges from $[-1, 1]$, where the sign indicates the direction of the linear relationship and the absolute value indicates its strength.

D.2 Categorical-Continuous: Correlation Ratio

The Correlation Ratio, η , measures the relationship between a categorical variable x and a continuous variable y . It is defined as the square root of the weighted variance of the category means divided by the total variance:

$$\eta = \sqrt{\frac{\sum_x n_x (\bar{y}_x - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (7)$$

where n_x is the number of samples in category x , $\bar{y}_x$ is the mean of y for samples in category x , and $\bar{y}$ is the overall mean. The value of η ranges from $[0, 1]$, with values closer to 1 indicating that the categorical variable provides more information about the continuous variable.

D.3 Categorical-Categorical: Cramér’s V

For two categorical variables with a contingency table of dimensions $r \times k$, Cramér’s V is based on the χ^2 statistic. To ensure robustness, we employ the bias correction proposed by Bergsma and Wicher (2013).

First, the bias-corrected ϕ^2 is calculated as:

$$\phi_{\text{corr}}^2 = \max \left(0, \frac{\chi^2}{n} - \frac{(k-1)(r-1)}{n-1} \right) \quad (8)$$

where n is the total number of samples. The corrected dimensions are:

$$r_{\text{corr}} = r - \frac{(r-1)^2}{n-1}, \quad k_{\text{corr}} = k - \frac{(k-1)^2}{n-1} \quad (9)$$

The final Cramér’s V score is then given by:

$$V_{\text{corr}} = \sqrt{\frac{\phi_{\text{corr}}^2}{\min(k_{\text{corr}} - 1, r_{\text{corr}} - 1)}} \quad (10)$$

The value ranges from $[0, 1]$, representing the strength of association between the two categorical variables.

E Complete Algorithm

Algorithm 1 provides the complete pseudocode for the StructSynth framework, integrating both the Evidence-Grounded Graph Induction and the Graph-Planned Conditional Synthesis stages.

F Detailed Experimental Setup

F.1 Datasets

The datasets utilized in this study are summarized in Table 4. They differ in tasks, number of features, and domain. We randomly split each dataset into training and test sets using an 8:2 ratio, yielding $\mathcal{D}_{\text{train-full}}$ and $\mathcal{D}_{\text{test}}$. To simulate data-scarce settings, we further subsample n instances from $\mathcal{D}_{\text{train-full}}$ to form the final training set, $\mathcal{D}_{\text{train}}$. This procedure is repeated ten times with different random seeds (42–51) to ensure robustness.

F.2 Baselines

For all baseline methods, with the exception of CuratedLLM, we utilize the implementations provided in the SynthCity (Qian et al., 2023) library. These models were run using their default hyperparameter configurations to ensure a standardized comparison. For CuratedLLM, we used the official source code provided by the authors, adapting it to our experimental setup. To ensure a fair comparison, it was configured with the same Large Language Model (LLM) settings as our proposed StructSynth, with detailed parameters provided in Table 5.

F.3 Implementation and Hyperparameters

The detailed hyperparameter configurations for the LLMs in StructSynth, the downstream evaluation models, and the Peter-Clark algorithm are provided in Table 5. For the software implementation, we utilize the Langchain library to manage LLM inference and the Causal-Learn library (Zheng et al., 2024) for the PC algorithm.

F.4 Evaluation Metrics

F.4.1 Downstream Model Performance

The Downstream Model Performance metric serves as the primary measure of the synthetic data’s practical utility. It evaluates whether the synthetic data, when used to augment the original training set, can improve the performance of a machine learning model on a real, unseen test set. For each generative method, we create an augmented training set, $\mathcal{D}_{\text{aug}}$, by combining the original limited training data, $\mathcal{D}_{\text{train}}$, with the generated synthetic data, $\mathcal{D}_{\text{synth}}$:

$$\mathcal{D}_{\text{aug}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{synth}} \quad (11)$$

A downstream machine learning model (in our experiments, an XGBoost model) is then trained exclusively on this augmented dataset, $\mathcal{D}_{\text{aug}}$, and evaluated on the held-out real test set, $\mathcal{D}_{\text{test}}$. For binary and multi-class classification tasks, we report the Area Under the Receiver Operating Characteristic Curve (AUC). For regression tasks, we report the R-squared (R^2) score.

F.4.2 Privacy Risk Assessment Metric

The Privacy Risk Assessment metric evaluates the extent to which a generative model overfits or memorizes the training data, $\mathcal{D}_{\text{train}}$. The complete set of real data instances is defined as $\mathcal{D}_{\text{real}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$. To ensure an unbiased comparison where the ideal probability is exactly 0.5, we first downsample the larger of the two sets ($\mathcal{D}_{\text{train}}$ or $\mathcal{D}_{\text{test}}$) to match the size of the smaller one.

For any two data points, x_i and x_j , from the mixed-type dataset, we define a distance function $d(x_i, x_j)$ as the sum of the L1 distance for numerical features and the Hamming distance for categorical features:

$$d(x_i, x_j) = \sum_{k \in \text{numerical}} |x_{i,k} - x_{j,k}| + \sum_{l \in \text{categorical}} \mathbb{I}(x_{i,l} \neq x_{j,l}) \quad (12)$$

For each synthetic sample $\tilde{x} \in \mathcal{D}_{\text{synth}}$, we identify its nearest neighbor (NN) from the entire set of real data instances, $\mathcal{D}_{\text{real}}$:

$$NN(\tilde{x}) = \arg \min_{x \in \mathcal{D}_{\text{real}}} d(\tilde{x}, x) \quad (13)$$

The Privacy Risk score is then calculated as the fraction of synthetic samples whose nearest neighbor is an instance from the training set $\mathcal{D}_{\text{train}}$:

$$\text{PrivacyRisk} = \frac{1}{|\mathcal{D}_{\text{synth}}|} \sum_{\tilde{x} \in \mathcal{D}_{\text{synth}}} \mathbb{I}(NN(\tilde{x}) \in \mathcal{D}_{\text{train}}) \quad (14)$$

Algorithm 1 StructSynth: Structure-Aware Tabular Data Synthesis

- 1: **Input:** Training data $\mathcal{D}_{\text{train}} = (X_{\text{train}}, A)$, number of synthetic samples s
2: **Output:** Synthetic dataset $\mathcal{D}_{\text{synth}} = (X_{\text{synth}}, A)$

Stage 1: Evidence-Grounded Graph Induction*1. Graph Initialization*

- 3: Select a few-shot subset $\mathcal{D}_{\text{few_shot}} \subseteq \mathcal{D}_{\text{train}}$
4: $V_{\text{source}} \leftarrow f_{\text{LLM}}(\pi_{\text{source}}(\mathcal{D}_{\text{few_shot}}))$
5: $G \leftarrow (V_{\text{source}}, \emptyset)$
6: $Q \leftarrow V_{\text{source}}$ ▷ Initialize queue for BFS
7: $V_{\text{visited}} \leftarrow \emptyset$

2. Iterative Expansion, Resolution, and Continuation (BFS)

- 8: **while** $Q \neq \emptyset$ **do**
 a. Expansion and Reasoned Link Generation
 9: $A_i \leftarrow \text{Dequeue}(Q)$
 10: $V_{\text{visited}} \leftarrow V_{\text{visited}} \cup \{A_i\}$
 11: Compute association scores $\mathcal{S}(A_i)$ between A_i and other attributes
 12: $P_i \leftarrow f_{\text{LLM}}(\pi_{\text{generate}}(A_i, G, \mathcal{S}(A_i), \mathcal{D}_{\text{few_shot}}))$
 13: $E_{\text{prop},t} \leftarrow \{(A_i \rightarrow A_j, r_{ij}) \mid (A_j, r_{ij}) \in P_i\}$
 14: $V_{\text{new},t} \leftarrow \{A_j \mid (A_j, _) \in P_i \wedge A_j \notin V\}$
 b. LLM-based Cycle Resolution
 15: $E_{\text{candidate}} \leftarrow E \cup E_{\text{prop},t}$
 16: $\Psi \leftarrow \text{DetectCycles}(V \cup V_{\text{new},t}, E_{\text{candidate}})$
 17: $E_{\text{pruned}} \leftarrow \emptyset$
 18: **for each** cycle $\psi \in \Psi$ **do**
 19: $e_{\text{to_remove}} \leftarrow f_{\text{LLM}}(\pi_{\text{resolve}}(\psi))$
 20: $E_{\text{pruned}} \leftarrow E_{\text{pruned}} \cup \{e_{\text{to_remove}}\}$
 21: **end for**
 c. Continuation of BFS
 22: $G \leftarrow (V \cup V_{\text{new},t}, E_{\text{candidate}} \setminus E_{\text{pruned}})$ ▷ Update graph
 23: Enqueue unvisited nodes from $V_{\text{new},t}$ into Q
24: **end while**
25: **Return** $G = (V, E)$

Stage 2: Graph-Planned Conditional Synthesis

- 26: Partition nodes V into topological layers $\mathcal{L} = (L_1, \dots, L_m)$
27: $X_{\text{synth}} \leftarrow \emptyset$
28: **for** $j \leftarrow 1$ to s **do**
 a. Generating Graph-Based Values
 29: $\tilde{x}_{j,V} \leftarrow \{\}$
 30: **for** $i \leftarrow 1$ to m **do**
 31: $L_i \leftarrow \mathcal{L}[i]$
 32: $\tilde{x}_{j,L_i} \leftarrow f_{\text{LLM}}(\pi_{\text{data_gen}}(\tilde{x}_{j,<i}, G[L_i \cup \Gamma^-(L_i)], L_i, \mathcal{D}_{\text{few_shot}}^i))$
 33: $\tilde{x}_{j,V} \leftarrow \tilde{x}_{j,V} \cup \tilde{x}_{j,L_i}$
 34: **end for**
 b. Generating Independent Values
 35: $A_{\text{iso}} \leftarrow A \setminus V$
 36: $\tilde{x}_{j,A_{\text{iso}}} \leftarrow f_{\text{LLM}}(\pi_{\text{data_gen_iso}}(\tilde{x}_{j,V}, A_{\text{iso}}, \mathcal{D}_{\text{few_shot}}^{\text{iso}}))$
 c. Final Assembly
 37: $\tilde{x}_j \leftarrow \text{concat}(\tilde{x}_{j,V}, \tilde{x}_{j,A_{\text{iso}}})$
 38: $X_{\text{synth}} \leftarrow X_{\text{synth}} \cup \{\tilde{x}_j\}$
39: **end for**
40: **Return** $\mathcal{D}_{\text{synth}} = (X_{\text{synth}}, A)$
-

Table 4: Summary of Dataset Characteristics.

Dataset	# Num Feat.	# Cat Feat.	# Classes	Domain	Task	Pre-LLM Knowledge Cutoff
Adult (Becker and Kohavi, 1996)	4	11	2	Social/Census	Binary	✗
Anxiety (Henderson et al., 2025)	12	7	3	Medical/Health	Multi-Class	✓
Compas (Angwin et al., 2016)	5	3	2	Crime	Binary	✗
Salary (Sajjad, 2025)	3	15	-	Economy	Regression	✓
Obesity (Palechor and de la Hoz Manotas, 2019)	7	8	-	Medical/Health	Regression	✗
Churn (Keramati et al., 2014)	8	4	2	Business	Binary	✗

Table 5: Hyperparameter Settings for Our Experiments.

Group	Hyperparameter Settings
LLMs	frequency_penalty = 0
	presence_penalty = 0
	top_p = 0.95
	temperature = 0.9
	max_tokens = 8000
XGBoost	booster = gbtrees
	n_estimators = 100
	learning_rate = 0.1
	max_depth = 6
	min_child_weight = 1
	gamma = 0
	subsample = 1
	colsample_bytree = 1
	reg_alpha = 0
	reg_lambda = 1
Random Forest	n_estimators = 200
	n_jobs = -1
Logistic Reg. / Ridge	max_iter = 1000 (Logistic Reg.)
	solver = 'lbfgs' (Logistic Reg.)
MLP	hidden_layer_sizes = (100,)
	max_iter = 500
PC algorithm	alpha = 0.05
	indep_test = 'kci'
	stable = True

A score approaching 0.5 suggests that the generative model has not memorized the training data.

F.4.3 Statistical Fidelity Metric

The Statistical Fidelity metric quantifies how well the synthetic data, $\mathcal{D}_{\text{synth}}$, preserves the pairwise statistical relationships present in the real data, $\mathcal{D}_{\text{real}}$. For each unique pair of columns (C_i, C_j) in the dataset, a difference score, $\Delta(C_i, C_j)$, is calculated. The calculation method varies for the three possible data type combinations:

- **Numerical-Numerical Pairs:** For two numer-

ical columns, the difference is the absolute difference between their Pearson correlation coefficients (ρ) in the real and synthetic datasets:

$$\Delta(C_i, C_j) = |\rho(C_{i,\text{real}}, C_{j,\text{real}}) - \rho(C_{i,\text{synth}}, C_{j,\text{synth}})| \quad (15)$$

- **Categorical-Categorical Pairs:** For two categorical columns, the difference is measured as the Total Variation Distance (TVD) of their joint probability distributions:

$$\Delta(C_i, C_j) = \frac{1}{2} \sum_{u,v} |P_{\text{real}}(C_i = u, C_j = v) - P_{\text{synth}}(C_i = u, C_j = v)| \quad (16)$$

where u and v iterate over all possible categories for columns C_i and C_j , respectively.

- **Numerical-Categorical Pairs:** For a mixed pair of a numerical column C_i and a categorical column C_j , the numerical column is first discretized into a categorical representation, C'_i , by binning its values into $q = 10$ quantiles. The difference is then calculated using the same TVD method as for categorical-categorical pairs:

$$\Delta(C_i, C_j) = \Delta(C'_i, C_j) \quad (17)$$

The overall Statistical Fidelity score is the mean of these difference scores computed over all unique pairs of columns in the dataset:

$$\text{StatisticalFidelity} = \frac{1}{\binom{K}{2}} \sum_{1 \leq i < j \leq K} \Delta(C_i, C_j) \quad (18)$$

where K is the total number of columns.

G Qualitative Graph Analysis

G.1 Structural Fidelity on Adult Dataset

To provide a qualitative illustration of our method's ability to preserve the core dependencies within the data in the absence of a ground-truth graph, we present a case study on the Adult dataset in Figure 6.

This visualization compares three distinct graphs: (a) a reference graph, established by running the PC algorithm on the complete dataset and refining the output with domain expertise; (b) the dependency graph discovered by StructSynth’s learning stage from $\mathcal{D}_{\text{train}}$; and (c) a graph re-discovered from $\mathcal{D}_{\text{synth}}$ generated by StructSynth using the PC algorithm.

A visual comparison reveals a strong alignment between our LLM-discovered graph (Figure 6b) and the reference graph (Figure 6a), particularly around the key *Salary* variable. This demonstrates the effectiveness of our structure learning phase in creating a high-quality dependency graph. More importantly, the remarkable structural similarity between the re-discovered graph (Figure 6c) and the reference graph serves as powerful evidence of our method’s fidelity. The ability of a standard discovery algorithm to independently recover the essential generative pathways from our synthetic data confirms that StructSynth effectively preserves the underlying dependency structure.

G.2 Structure Discovery Comparisons on Asia Dataset

To provide a concrete illustration of the structural discovery capabilities discussed above, Figure 7 presents a visual comparison of the dependency graphs learned by StructSynth and four representative baselines on the Asia dataset ($n = 100$). Although the Asia benchmark is defined as a causal Bayesian network, StructSynth uses the ground-truth DAG only as a reference directed dependency structure for SHD evaluation. The **Ground Truth** graph represents the reference dependency structure of the dataset, where, for example, *smoke* is linked to *lung* (cancer) and *bronc* (bronchitis), which in turn are linked to *dysp* (dyspnoea).

StructSynth successfully recovers the majority of these key dependencies, including the V-structure at *dysp* and the dependency chain from *smoke* to *lung*. The slight deviations are minimal and do not disrupt the global topological ordering, ensuring that the subsequent generation process follows a valid dependency path.

In contrast, baseline methods exhibit significant structural distortions:

- **GraDe** tends to produce an overly dense graph with numerous spurious edges (e.g., direct links between *asia* and *dysp*), likely due to its reliance

on attention maps that can conflate indirect associations with direct dependencies.

- **GOGGLE**, while capturing some dependencies, suffers from edge reversals (e.g., *lung* $\rightarrow$ *smoke*) and misses critical connections (e.g., *bronc* is isolated from *dysp*). This reflects the instability of end-to-end differentiable structure learning in low-data regimes.
- **NoTears** yields a sparse and fragmented graph, failing to identify the relationship between *smoke* and *lung*, which are central to the dataset’s dependency structure.
- **FCI**, a constraint-based method, correctly identifies some skeletons but leaves many edges undirected (represented by bi-directed or unoriented edges), failing to provide a clear generative order.

These visual examples corroborate our quantitative findings in Figure 3, confirming that StructSynth’s LLM-guided approach offers superior robustness in discovering interpretable and accurate dependency structures compared to both traditional and deep learning-based alternatives.

H Sensitivity to Downstream Model Choice

To verify that the gains reported in the main experiments are not specific to a single evaluation model, we compare StructSynth and CLLM under four downstream learners—XGBoost (tree-based), Random Forest (ensemble), Logistic Regression / Ridge (linear), and MLP (neural)—on four representative datasets spanning classification, regression, and post-cutoff settings (Table 6). StructSynth wins 14 out of 16 cells, outperforming both CLLM and the train-only baseline across all three classification datasets under every evaluation model. The two exceptions occur on the Salary regression task, where Ridge and MLP favor $\mathcal{D}_{\text{train}}$ and CLLM respectively, suggesting that linear and shallow neural models are more sensitive to distributional shifts in regression settings. Across classification datasets, the margin is largest for MLP (+3.26 AUC on Adult), indicating that neural models benefit most from the improved distributional quality of StructSynth’s synthetic data.

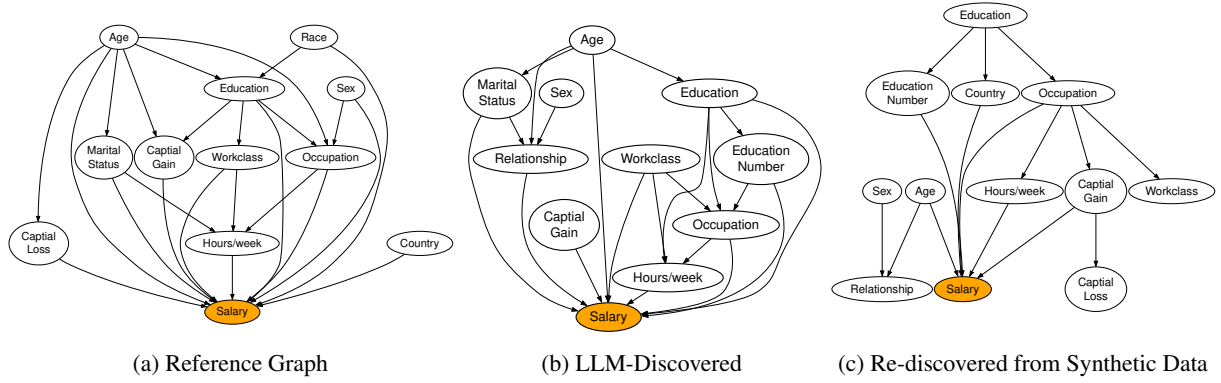


Figure 6: Visual Analysis of Structural Fidelity on the Adult Dataset with *Salary* as the Label Column.

Table 6: Downstream performance (%) with different evaluation models on four representative datasets. (C): AUC, (R): R^2 . Best per cell in **bold**.

Dataset	Eval. Model	$\mathcal{D}_{\text{train}}$	CLLM	StructSynth
Adult (C)	XGBoost	82.51	83.95	85.55
	Random Forest	85.03	84.63	87.45
	Logistic Reg.	85.68	85.49	87.80
	MLP	83.63	83.59	86.85
Anxiety [†] (C)	XGBoost	85.49	85.17	86.45
	Random Forest	86.85	85.14	87.04
	Logistic Reg.	87.26	85.87	87.82
Salary [†] (R)	MLP	84.85	82.27	85.76
	XGBoost	49.71	54.53	55.98
	Random Forest	55.16	51.29	57.12
Churn (C)	Ridge	59.48	52.58	56.80
	MLP	44.61	46.00	42.64
	XGBoost	86.86	89.81	90.39
Churn (C)	Random Forest	90.78	91.66	92.49
	Logistic Reg.	90.61	91.91	92.19
	MLP	91.71	92.36	93.61

I Efficiency Analysis

A practical concern is the additional cost introduced by structure discovery. Table 7 reports the token consumption of CLLM and StructSynth under the **100-shot** setting, averaged over six real-world datasets. Overall, introducing the graph-based structure modeling in StructSynth does *not* lead to a prohibitive cost increase: the average total usage rises from **219.8K** tokens (CLLM total) to **287.9K** tokens (StructSynth combined total), corresponding to an absolute overhead of **+68.1K** tokens (about **+31.0%**). This overhead is relatively stable across datasets (roughly **+19%** to **+42%** in total), suggesting that the additional structure reasoning introduces a consistent but moderate computational budget.

A key observation is that the overhead is **highly skewed toward the input side**. Concretely, StructSynth increases average *input* tokens by **+65.1K (+57.1%)**, while the *output* tokens only

increase by **+2.9K (+2.7%)**. This pattern is expected: StructSynth needs additional prompt context to (i) elicit and refine the dependency graph and (ii) condition the synthesis stage on the discovered structure, whereas the corresponding model responses (e.g., edge lists / compact graph representations) are short and the final table synthesis outputs remain comparable in length to CLLM.

Importantly, this “input-dominant” overhead implies that the **real monetary overhead is often smaller than what the raw total-token increase might suggest**, because in many pricing schemes output tokens are priced higher than input tokens. Taken together, the token analysis indicates that StructSynth achieves structure-aware generation with a **modest and practically manageable** cost increase, largely attributable to additional prompting rather than longer model generations.

J Extended Discussion

In this section, we discuss the broader implications of our results, analyze key design decisions, and contextualize StructSynth within the landscape of structure-aware generative methods.

J.1 Structure-Blind vs. Structure-Aware Generation

A central question raised by our work is whether explicit structural guidance provides a *qualitative*—rather than merely quantitative—advantage over structure-blind LLM generation. To address this, we compare StructSynth against its closest structure-blind counterpart, CLLM, which uses the same LLM with identical hyperparameters but generates data without any structural generation plan.

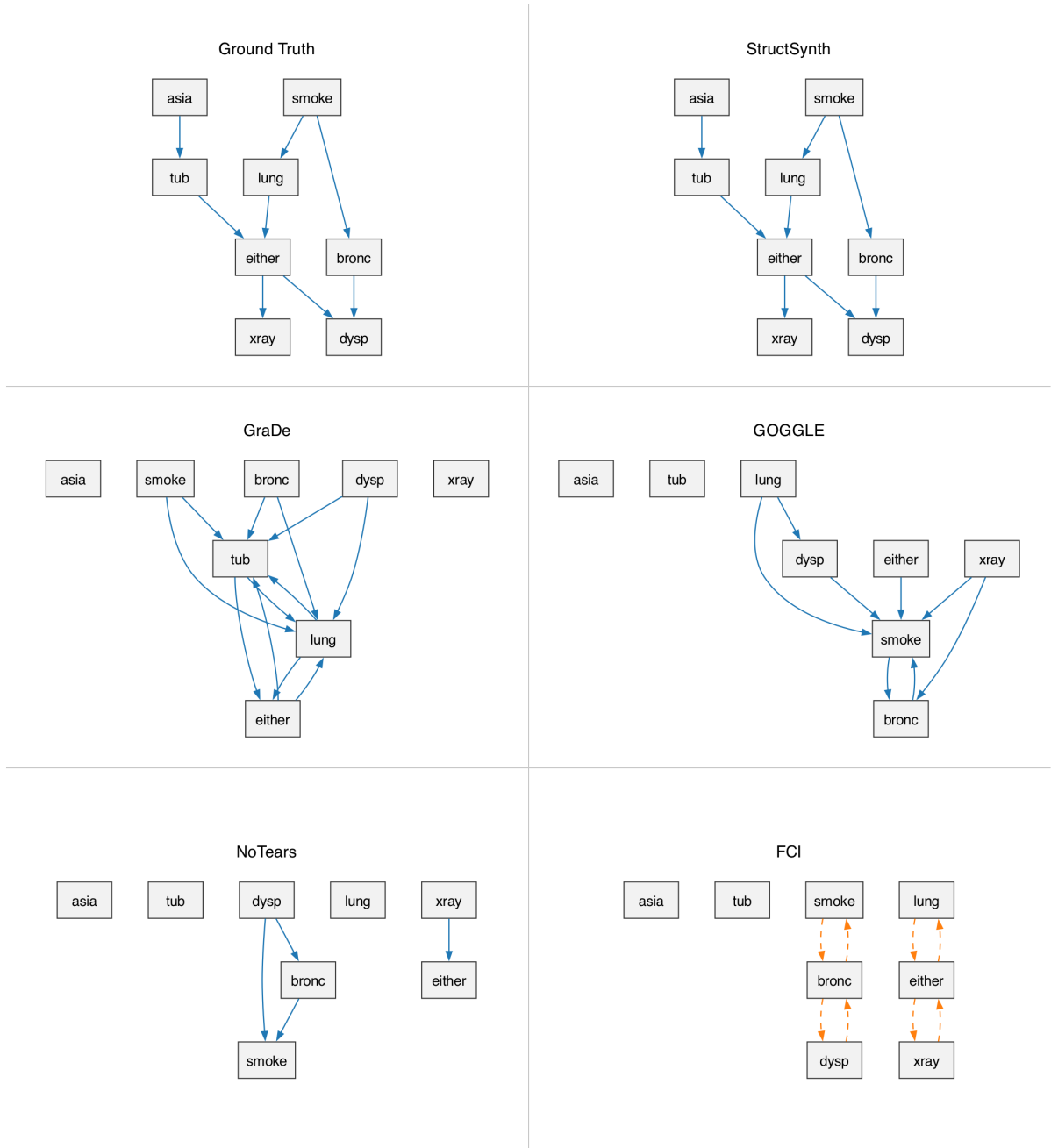


Figure 7: Comparison of discovered dependency structures on the Asia dataset ($n = 100$). **Ground Truth**: The reference DAG from the bnlearn repository. **StructSynth**: Our method closely recovers the ground truth. **GraDe**, **GOGGLE**, **NoTears**, **FCI**: Baseline methods exhibit varying degrees of structural errors, including missing edges, reversed edges, and spurious connections.

Our ablation study (Table 3) provides direct evidence for three distinct dimensions of improvement. First, regarding **structural modeling**: CLLM processes features in an arbitrary serialized order, implicitly inducing dependencies from the input sequence. In contrast, StructSynth encodes dependencies as an explicit DAG, ensuring that relationships are *discovered* rather than incidentally induced. Second, regarding **generation strategy**:

CLLM generates all features in a single pass without constraints, whereas StructSynth follows a topological ordering so that each feature is conditioned on its discovered parent nodes. Removing this ordering alone (*No Topological Order* variant) degrades AUC by 1.1 points, while removing the graph entirely (*No Structure* variant, equivalent to CLLM) causes a 1.6-point drop. Third, regarding **regularization**: the DAG serves as a structural

Table 7: Token usage under 100-shot ($\times 10^3$ tokens). We report input/output tokens for CLLM and StructSynth; overhead compares StructSynth combined totals (graph+table) against CLLM totals.

Method	Component	Adult	Anxiety	Churn	Compas	Obesity	Salary	Avg.
CLLM	Input	40.6	142.2	38.3	26.5	70.7	366.1	114.1
	Output	105.2	175.2	102.6	79.5	13.5	158.6	105.8
	Total	145.8	317.4	140.9	106.0	84.2	524.7	219.8
StructSynth	Graph Input	38.1	89.0	41.4	17.1	43.5	111.1	56.7
	Graph Output	1.5	3.1	2.2	1.0	1.7	3.1	2.1
	Graph total	39.6	92.1	43.6	18.1	45.3	114.3	58.8
	Table Input	52.7	168.3	46.0	32.3	61.1	374.3	122.5
	Table Output	114.3	177.4	102.7	75.9	12.0	157.1	106.6
	Table total	167.0	345.7	148.7	108.2	73.2	531.4	229.0
	Combined total	206.6	437.8	192.3	126.3	118.4	645.7	287.9
Token overhead of StructSynth vs. CLLM (Combined – Total)								
	Δ Input (K)	50.2	115.1	49.1	22.9	33.9	119.3	65.1
	Δ Output (K)	10.6	5.3	2.3	−2.6	0.2	1.6	2.9
	Δ Total (K)	60.8	120.4	51.4	20.3	34.2	121.0	68.1
	Relative Input (%)	123.6	80.9	128.2	86.4	48.0	32.6	57.1
	Relative Output (%)	10.1	3.0	2.2	−3.3	1.5	1.0	2.7
	Relative Total (%)	41.7	37.9	36.5	19.2	40.6	23.1	31.0

regularizer that constrains the generation space, preventing the LLM from overfitting to spurious patterns in the few available training samples. This regularization effect is evidenced by StructSynth’s superior privacy preservation (avg. rank 1.50 vs. CLLM’s 3.50), indicating reduced memorization of training records.

These results demonstrate that the advantage of StructSynth is *not* merely the result of adding peripheral components to an LLM pipeline, but stems from a fundamental shift in how dependencies are modeled and enforced during generation.

J.2 Understanding the Fidelity-Privacy Trade-off

A nuanced finding in our evaluation is that StructSynth does not achieve the highest Statistical Fidelity score (avg. rank 8.50), despite its strong downstream utility (avg. rank 1.00) and privacy preservation (avg. rank 1.50). We argue that this apparent discrepancy is both expected and desirable, reflecting the inherent tension between fidelity and privacy.

To understand this trade-off, consider GReaT, which achieves strong Statistical Fidelity (avg. rank 4.00). However, its Privacy Risk score of 90.15% reveals severe memorization—the model essentially reproduces training records rather than

generating novel samples. This high fidelity is therefore an artifact of overfitting rather than genuine distributional learning. At the other extreme, the Bayesian Sampler in our ablation achieves the best raw fidelity score (48.86) but the lowest AUC (81.17), conclusively demonstrating that **high pairwise fidelity does not guarantee downstream utility**.

This discrepancy can be further understood by examining what Statistical Fidelity actually measures. The metric quantifies the preservation of *pairwise* correlations across all feature pairs. In contrast, StructSynth’s DAG encodes *conditional* dependencies—the probabilistic relationships between parent and child nodes. These conditional dependencies are what drive downstream model performance, as they capture the generative mechanisms underlying the data. StructSynth optimizes for preserving these conditional structures rather than pairwise statistics, which explains its superior utility despite moderate pairwise fidelity scores.

In summary, StructSynth’s generation plan acts as a *principled regularizer*: it constrains the generation process to follow discovered conditional dependencies, which simultaneously prevents memorization (ensuring privacy) and preserves the dependency structure that matters most for downstream tasks (ensuring utility), even at

the cost of slightly lower pairwise correlation matching.

J.3 When Does Structural Guidance Help Most?

Our experiments on varying training sample sizes (Figure 4) reveal that StructSynth’s advantage is most pronounced in the low-data regime ($n \leq 50$), where it maintains high AUC while baselines deteriorate significantly. As n increases toward 200, the performance gap narrows, though StructSynth remains competitive.

This pattern has an intuitive explanation. When data is scarce, statistical methods for structure discovery (e.g., PC algorithm, NoTears) suffer from unstable association estimates, leading to unreliable graphs. In this regime, StructSynth’s LLM-guided discovery provides a powerful complement: the model’s semantic prior knowledge acts as a *structural regularizer*, stabilizing the discovery process by filtering out statistically spurious correlations that would mislead purely data-driven approaches. As more data becomes available, statistical methods become increasingly reliable on their own, diminishing the relative advantage of LLM-guided reasoning.

This analysis also reveals StructSynth’s natural boundary conditions. The method is best suited for scenarios where: (1) the number of training samples is severely limited ($n \leq 100$); (2) features have semantically meaningful names and relationships that an LLM can reason about; and (3) the underlying dependency structure is sparse enough to be well-represented by a DAG. In contrast, for purely numerical datasets with opaque feature names (e.g., anonymized sensor readings), the LLM’s semantic prior provides less value, and the framework may offer diminished advantages over purely statistical approaches.

J.4 Relationship to Causal Inference

It is important to clearly distinguish StructSynth’s dependency graphs from causal graphs in the traditional sense. As established in Appendix C, our DAG serves as a *generative dependency structure*—its edges encode statistical and probabilistic dependencies used to organize the synthesis process, not claims of ground-truth causality.

This distinction separates StructSynth from classical causal discovery methods such as the PC algorithm (Spirtes et al., 2000), FCI (Spirtes et al., 2000), or GES (Chickering, 2002), which aim to

identify the true causal data-generating mechanism under specific assumptions (e.g., faithfulness, sufficiency). Our objective is fundamentally different: we seek a dependency structure that enables *effective conditional generation*, not one that supports causal interventions or counterfactual reasoning.

Nevertheless, our SHD evaluations on benchmark DAGs (Figure 3) demonstrate that the structures discovered by StructSynth often closely approximate the ground-truth causal graphs. This is not a contradiction: when the true data-generating process follows a DAG, a well-discovered dependency structure will naturally align with the causal structure, even though causal identification is not our explicit goal. This alignment is a desirable side effect that contributes to the high fidelity of our generated data.

K Detailed Prompts for StructSynth

This section provides a guide to the prompt templates used in the StructSynth framework. A key step in our methodology is the textualization of diverse data structures—including statistical scores, dependency graphs, and tabular data—to serve as inputs for the Large Language Model (LLM). Box 1 shows concrete examples of this process.

The StructSynth pipeline employs a sequence of prompts to complete its tasks. First, the π_{source} prompt initiates graph discovery by identifying initial root nodes. During the iterative graph construction, the π_{generate} prompt proposes new dependency edges with corresponding rationales. If a cycle is detected, the π_{resolve} prompt is used to analyze the conflict and remove the weakest link. Finally, for data synthesis, the $\pi_{\text{data_gen}}$ prompt generates values for dependent features, after which the π_{iso} prompt generates values for the remaining isolated features.

Examples of Data Textualization for LLM Prompts

1. Textual Representation of Statistical Evidence (Association Scores):

Association scores for 'Education', scaled 0 to 1.

(Levels: >0.8 Very High, >0.6 High, >0.4 Moderate, >0.2 Low, <=0.2 Very Low)

- Occupation: 0.85 (Very High)
- Salary: 0.72 (High)
- Marital-Status: 0.55 (Moderate)
- Hours-per-week: 0.31 (Low)
- Sex: 0.12 (Very Low)

2. Textual Representation of a Dependency Graph:

Current dependency graph structure with rationales:

- Age -> Marital-Status

Rational: A person's age is a primary factor influencing their marital status.

- Sex -> Relationship:

Rational: Gender often determines the relationship role within a household.

- Country -> Race:

Rational: Native country has a strong statistical correlation with a person's race.

- Race -> Education:

Rational: Statistical data often shows a correlation between a person's race and their level of educational attainment.

3. Textual Representation of Tabular Data (Markdown):

age	education	occupation	sex	hours-per-week	salary
39	Bachelors	Adm-clerical	Male	40	<=50K
50	Bachelors	Exec-managerial	Male	13	<=50K
28	Assoc-acdm	Prof-specialty	Female	45	<=50K

Prompt for Source Nodes Initialization of Tabular Data (π_{source})

You are an expert data analyst. Based on the following information:

Task Description:

Identify the initial "source nodes" for a dependency graph. A source node is a root attribute that is generated without conditioning on other attributes in this dataset.

All Feature Descriptions:

{all_features}

Example Data:

{few_shot_data}

Your Task is to Identify the Source Nodes:

- Your only task is to analyze the provided column descriptions and data.
- Identify the features that are most likely to be **source nodes** (i.e., fundamental attributes that are not dependent on other features).
- Provide your answer as a simple list of the identified source node names.

Prompt for Link Generation in Dependency Discovery (π_{generate})

You are an expert data analyst building a dependency graph. Based on the following information:

Task Description:

For the given {current_feature}, your task is to propose its direct successors (dependents) from the list of available features. For each proposed dependency, you must provide a brief, evidence-based rationale.

Current Graph State:

{structure_graph}

All Candidate Features:

{all_features}

Statistical Evidence (Association Scores):

{association_scores}

Example Data:

{few_shot_data}

Your Task is to Propose and Justify New Links:

- Your focus is on the feature: {current_feature}.
- Review the **Statistical Evidence**, which shows the strength of association between {current_feature} and other candidate features {all_features}.
- Based on the statistical scores, the example data, and the existing graph, identify which other features are most likely to be **direct dependents** of {current_feature}.
- For each dependency you propose, you **must** provide a concise rationale. This rationale should explain why the dependency makes sense (e.g., "Higher Education is strongly correlated with and typically precedes higher Income").
- Do not propose links that would create an obvious logical contradiction with the existing graph structure.
- Format your final output as a list of dependency edges and their corresponding rationales.

Prompt for Cycle Resolution in Dependency Discovery (π_{resolve})

You are a logical reasoning expert tasked with resolving a contradiction in a dependency graph. Based on the following information:

Task Description:

Analyze the provided dependencies and their rationales within the cycle. Identify the single dependency with the weakest or least plausible rationale and recommend it for removal to break the cycle.

Conflicting Cycle with Rationales:

{cycle_with_rationales}

Your Task is to Identify the Weakest Link:

- A logical contradiction (a cycle) has been detected in the graph, detailed in **Conflicting Cycle with Rationales**.
- Your task is to carefully evaluate the rationale provided for each edge in this cycle.
- Based on your analysis of the rationales and any relevant context from the column descriptions or data, identify the **single weakest link**.
- The weakest link is the dependency whose rationale is the least plausible, least supported, or most likely to be spurious.
- Provide your final answer as the single dependency edge that should be removed to resolve the contradiction.

Prompt for Graph-Planned Conditional Synthesis ($\pi_{\text{data_gen}}$)

You are a helpful AI assistant that generates realistic tabular data based on structural dependencies.

Task Description:

Generate a realistic synthetic table containing exactly the columns listed in {features_to_generate}. This generation must be conditioned on the provided values of their parent features.

Conditioning Data (Current Results):

{current_result}

Relevant Dependency Structure:

{related_structure_graph}

Example Data:

{related_few_shot_data}

Your Task is to Generate the Synthetic Data:

- Your primary task is to generate realistic data for the features listed in {features_to_generate}.
- This generation must be **conditioned** on the data provided in **Conditioning Data (Current Results)**. These are the values of the parent nodes that influence the features you are generating.
- The **Relevant Dependency Structure** shows you exactly how the conditioning features relate to the features you need to generate.
- Use the **Example Data** to understand the format, range, and statistical properties of the real data.
- The generated table should contain columns for exactly the features listed in {features_to_generate}.
- Ensure the generated values are plausible and respect the learned dependencies.

Prompt for Independent Feature Synthesis ($\pi_{\text{data_gen_iso}}$)

You are a helpful AI assistant that completes tabular data records by generating values for remaining features.

Task Description:

The core, structurally-dependent features of a dataset have already been generated. Your task is to generate plausible values for the remaining isolated features, ensuring they are statistically consistent with the core features.

Conditioning Data (Generated Graph-Based Features):

{graph_based_values}

Features to Generate (Isolated Features):

{isolated_features_to_generate}

Example Data:

{related_few_shot_data}

Your Task is to Generate the Independent Values:

- Your primary task is to generate realistic data for the features listed in {isolated_features_to_generate}.
- This generation must be **conditioned** on the complete set of core features provided in **Conditioning Data**.
- The features you are generating do not have direct parent-child dependencies in the learned graph, but their values should still be plausible and consistent in the context of the entire data record.
- Use the **Example Data** to understand the typical format and statistical properties of these isolated features.
- The generated table should contain columns for exactly the features listed in {isolated_features_to_generate}.